arXiv:2603.05451v1 [cs.CL] 5 Mar 2026

FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling

Ted Zadouri[*1,6] Markus Hoehnerbach[*2] Jay Shah[*3]
Timmy Liu[4] Vijay Thakkar[2,5] Tri Dao[1,6]
[1]Princeton University [2]Meta [3]Colfax Research [4]NVIDIA [5]Georgia Tech [6]Together AI

Abstract

Attention, as a core layer of the ubiquitous Transformer architecture, is the bottleneck for large language models and long-context applications. While FlashAttention-3 optimized attention for Hopper GPUs through asynchronous execution and warp specialization, it primarily targets the H100 architecture. The AI industry has rapidly transitioned to deploying Blackwell-based systems such as the B200 and GB200, which exhibit fundamentally different performance characteristics due to asymmetric hardware scaling: tensor core throughput doubles while other functional units (shared memory bandwidth, exponential units) scale more slowly or remain unchanged. We develop several techniques to address these shifting bottlenecks on Blackwell GPUs: (1) redesigned pipelines that exploit fully asynchronous MMA operations and larger tile sizes, (2) software-emulated exponential and conditional softmax rescaling that reduces non-matmul operations, and (3) leveraging tensor memory and the 2-CTA MMA mode to reduce shared memory traffic and atomic adds in the backward pass. We demonstrate that our method, FlashAttention-4, achieves up to 1.3× speedup over cuDNN 9.13 and 2.7× over Triton on B200 GPUs with BF16, reaching up to 1613 TFLOPs/s (71% utilization). Beyond algorithmic innovations, we implement FlashAttention-4 entirely in CuTe-DSL embedded in Python, achieving 20-30× faster compile times compared to traditional C++ template-based approaches while maintaining full expressivity.

1 Introduction

The Transformer architecture [27] continues to serve as the primary backbone for nearly all AI applications, from large language models [2] to vision [8] and multimodal systems. For Transformers, the attention mechanism constitutes the primary computational bottleneck, with self-attention scores computed between queries and keys exhibiting quadratic scaling in sequence length. Scaling attention to longer contexts unlocks new capabilities such as reasoning over multiple documents [10, 24], modeling entire codebases [22], and processing high-resolution videos [3, 11]. Meanwhile, accelerator hardware continues to evolve rapidly [19], with each generation delivering substantially higher peak compute throughput. However, this evolution is asymmetric: while matrix multiplication units scale aggressively, other functional units such as memory bandwidth and specialized compute units scale more slowly, creating increasingly unbalanced hardware pipelines that demand careful algorithmic co-design.

This has generated sustained interest in making attention faster through algorithmic innovations that deeply integrate knowledge of GPU hardware characteristics. Dao et al. [6] introduced FlashAttention, which eliminates intermediate reads/writes to slow global memory through

[*]Equal contribution

arXiv:2603.05451v1 [cs.CL] 5 Mar 2026

FlashAttention-4：面向非对称硬件扩展的算法与内核流水线协同设计

Ted Zadouri[*1、 6] MarkusHoehnerbach[*2] JayShah[*3]TimmyLiu[4] Vijay Thakkar [2,5] Tri Dao[1,6]普林斯顿大学 [2]Meta [3]Colfax Research {v25>NVIDIA {v26>乔治亚理工学院 {v27>Together AI

摘要

Attention, as a core layer of the ubiquitous Transformer architecture, is the bottleneck for large language models and long-context applications. While FlashAttention-3 optimized attention for Hopper GPUs through asynchronous execution and warp specialization, it primarily targets the H100 architecture. The AI industry has rapidly transitioned to deploying Blackwell-based systems such as the B200 and GB200, which exhibit fundamentally different performance characteristics due to asymmetric hardware scaling: tensor core throughput doubles while other functional units (shared memory bandwidth, exponential units) scale more slowly or remain unchanged. We develop several techniques to address these shifting bottlenecks on Blackwell GPUs: (1) redesigned pipelines that exploit fully asynchronous MMA operations and larger tile sizes, (2) software-emulated exponential and conditional softmax rescaling that reduces non-matmul operations, and (3) leveraging tensor memory and the 2-CTA MMA mode to reduce shared memory traffic and atomic adds in the backward pass. We demonstrate that our method, FlashAttention-4, achieves up to 1.3× speedup over cuDNN 9.13 and 2.7× over Triton on B200 GPUs with BF16, reaching up to 1613 TFLOPs/s (71% utilization). Beyond algorithmic innovations, we implement FlashAttention-4 entirely in CuTe-DSL embedded in Python, achieving 20-30× faster compile times compared to traditional C++ template-based approaches while maintaining full expressivity.

1 引言

Transformer架构 [27] 至今仍几乎是所有AI应用的核心基础，从大型语言模型 [2]到视觉 [8]及多模态系统皆如此。对于Transformer而言，注意力机制构成了主要的计算瓶颈——查询项与键项之间计算得出的自注意力分数会随着序列长度的增加而呈二次方级增长。若能提升处理长上下文时的注意力效率，就能实现诸多新功能，比如对多份文档进行推理 [10, 24],、对整个代码库进行建模 [22], 以及处理高分辨率视频 [3, 11]。与此同时，加速器硬件也在持续快速发展 [19], ，每一代产品的峰值计算吞吐量都大幅提升。然而这种发展并非对称的：虽然矩阵乘法单元的性能提升极为显著，但内存带宽和专用计算单元等其他功能单元的提升速度则较慢，这导致了日益失衡的硬件架构，因而需要通过精细的算法协同设计来加以解决。

这引发了人们持续的兴趣，希望借助深入掌握GPU硬件特性的算法创新来提升注意力机制的处理速度。Dao等人 [6] 提出了FlashAttention，该方案通过消除对速度较慢的全局内存的中间读写操作来实现这一目标

[*]同等贡献

novel tiling and kernel fusion. Dao [5] restructured this as FlashAttention-2 to parallelize over the sequence length dimension, improving GPU occupancy. Shah et al. [23] further adapted the algorithm for Hopper GPUs as FlashAttention-3, exploiting asynchronous execution through warp specialization and incorporating FP8 support. Recent work has also explored low-precision attention: SageAttention [13] achieves speedups through INT8 quantization, SageAttention2 [12] extends this with INT4/FP8 quantization, and SageAttention3 [14] demonstrates FP4 quantization on Blackwell consumer GPUs. However, these approaches primarily target consumer GPUs, while most AI compute is deployed on datacenter GPUs. Meanwhile, FlashAttention-3 primarily targets the NVIDIA Hopper H100 architecture, while the AI industry has rapidly transitioned to deploying Blackwell-based datacenter systems [19] such as the B200 and GB200, which represent a new generation of GPUs with fundamentally different performance characteristics.

A critical trend in accelerator evolution is the asymmetric scaling of hardware units. Although Blackwell B200 doubles the tensor core throughput compared to Hopper H100 (2.25 PFLOPS vs. 1 PFLOPS for FP16/BF16), other functional units (shared memory bandwidth, exponential units, and integer/floating point ALUs) scale more slowly or remain unchanged. As a result, non-MMA resources emerge as bottlenecks. Our roofline analysis (Sections 3.1 and 3.2) reveals that for typical attention workloads on Blackwell, surprisingly, shared memory traffic and exponential operations now dominate execution time, exceeding MMA compute by 25-60%. Additionally, Blackwell introduces new architectural features: 256 KB of tensor memory (TMEM) per SM for storing intermediate tensor core results, 128×128 MMA tiles (double the area of Hopper's 64×128), and fully asynchronous tensor core operations that write directly to TMEM. Simply porting existing attention algorithms to this new hardware either leaves significant performance on the table or is impossible due to lack of forward compatibility for Hopper MMA instructions.

To this end, we propose FlashAttention-4, which co-designs the algorithm and kernel implementation to address the shifting bottlenecks in modern GPU architectures. Rather than treating hardware as a uniform compute resource, we explicitly identify and mitigate bottlenecks in non-matmul units through algorithmic innovations:

1. **Redesigned pipeline for maximum overlap:** We develop new software pipelines for both forward and backward passes that exploit Blackwell's fully asynchronous MMA operations and larger tile sizes to maximize overlap between tensor cores, softmax computation, and memory operations.
2. **Exponential unit bottleneck mitigation:** For the forward pass, we implement software-emulated exponential functions using polynomial approximation on FMA units, increasing exponential throughput. We also introduce conditional softmax rescaling that skips unnecessary rescaling operations.
3. **Shared memory traffic reduction:** For the backward pass, we leverage tensor memory to store more intermediate results, reducing shared memory traffic. We also leverage Blackwell's 2-CTA MMA mode, so each CTA stages and loads half of operand B to further reduce shared memory traffic, which we exploit to restructure the dQ step to halve the number of atomic reductions. We also implement a deterministic execution mode with minimal performance overhead, enabling reproducible training for reinforcement learning applications.
4. **Improved scheduling and resource allocation:** We develop new CTA scheduling strategies and register allocation schemes tailored to Blackwell's resource constraints and larger tile sizes.

Beyond algorithmic innovations, we implement FlashAttention-4 entirely in CuTe-DSL embedded in Python, achieving 20-30× faster compile times compared to traditional C++ template-based approaches while maintaining full expressivity. This framework significantly improves developer

novel tiling and kernel fusion. Dao [5] restructured this as FlashAttention-2 to parallelize over the sequence length dimension, improving GPU occupancy. Shah et al. [23] further adapted the algorithm for Hopper GPUs as FlashAttention-3, exploiting asynchronous execution through warp specialization and incorporating FP8 support. Recent work has also explored low-precision attention: SageAttention [13] achieves speedups through INT8 quantization, SageAttention2 [12] extends this with INT4/FP8 quantization, and SageAttention3 [14] demonstrates FP4 quantization on Blackwell consumer GPUs. However, these approaches primarily target consumer GPUs, while most AI compute is deployed on datacenter GPUs. Meanwhile, FlashAttention-3 primarily targets the NVIDIA Hopper H100 architecture, while the AI industry has rapidly transitioned to deploying Blackwell-based datacenter systems [19] such as the B200 and GB200, which represent a new generation of GPUs with fundamentally different performance characteristics.

A critical trend in accelerator evolution is the asymmetric scaling of hardware units. Although Blackwell B200 doubles the tensor core throughput compared to Hopper H100 (2.25 PFLOPS vs. 1 PFLOPS for FP16/BF16), other functional units (shared memory bandwidth, exponential units, and integer/floating point ALUs) scale more slowly or remain unchanged. As a result, non-MMA resources emerge as bottlenecks. Our roofline analysis (Sections 3.1 and 3.2) reveals that for typical attention workloads on Blackwell, surprisingly, shared memory traffic and exponential operations now dominate execution time, exceeding MMA compute by 25-60%. Additionally, Blackwell introduces new architectural features: 256 KB of tensor memory (TMEM) per SM for storing intermediate tensor core results, 128×128 MMA tiles (double the area of Hopper's 64×128), and fully asynchronous tensor core operations that write directly to TMEM. Simply porting existing attention algorithms to this new hardware either leaves significant performance on the table or is impossible due to lack of forward compatibility for Hopper MMA instructions.

为此，我们提出了 FlashAttention-4，该方案通过协同设计算法与内核实现方式，有效应对现代GPU架构中不断变化的瓶颈问题。我们并未将硬件视为统一的计算资源，而是借助算法创新，有针对性地识别并消除非矩阵乘法单元中的瓶颈。

1. **为实现最大程度重叠而重新设计的流水线：**我们为前向传播和后向传播分别开发了新的软件流水线，充分利用Blackwell架构的全异步多精度算术单元以及更大的分块尺寸，从而最大化张量核心、softmax函数计算与内存操作之间的重叠度。

2. **指数运算单元瓶颈缓解：**在前向传播过程中，我们利用FMA单元的多项式近似技术来实现软件模拟的指数函数，从而提升指数运算的处理效率。同时，我们还引入了条件式的softmax函数重新缩放机制，以避免不必要的重新缩放操作。

3. **共享内存流量降低：**在开展后向传播时，我们利用张量内存来存储更多的中间结果，从而减少共享内存流量。此外，我们还采用了Blackwell架构的2-CTA模式下的多精度算术单元，使得每个CTA阶段仅加载操作数B的一半，进一步降低共享内存流量；同时我们借助这一特性重构了dQ计算步骤，将原子化减法操作的次数减少一半。另外，我们还实现了一种性能开销极小的确定性执行模式，为强化学习应用提供可复现的训练环境。

4. **改进的调度机制与寄存器分配方案：**我们针对Blackwell的资源限制以及更大的分块尺寸，设计了全新的CTA调度算法与寄存器分配方案。

在算法创新之外，我们完全采用嵌入在Python语言中的CuTe-DSL语言来实现相关功能，相较于传统的基于C++ 模板的方法，其编译速度提升了20-30× 倍，同时还保持了极高的表达能力。该框架大幅提升了开发人员的

productivity and lowers the barrier to entry, enabling researchers to rapidly prototype and deploy new attention variants without deep expertise in C++ template metaprogramming.

To empirically validate our method, we benchmark FlashAttention-4 on the B200 GPU and show that (1) BF16 achieves up to 1.3× speedup over cuDNN and 2.7× over the Triton implementation, (2) we achieve near-peak utilization on the shifted bottleneck resources, reaching up to ∼1600 TFLOPS (71% theoretical max), and (3) for large sequence lengths, FlashAttention-4 outperforms alternative attention implementations.

We open source FlashAttention-4 with a permissive license and are working to integrate it with popular libraries to benefit the largest number of researchers and developers. The code is available at `https://github.com/Dao-AILab/flash-attention/tree/main/flash_attn/cute`

2 Background

2.1 Multi-Head Attention

Let $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N\times d}$ be the query, key and value input sequences associated to a single head, where N is the sequence length and d is the head dimension. The attention output $\mathbf{O} \in \mathbb{R}^{N\times d}$ is computed as:

$$\begin{aligned}\mathbf{S} &= \alpha\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N\times N},\\ \mathbf{P} &= \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N\times N},\\ \mathbf{O} &= \mathbf{P}\mathbf{V} \in \mathbb{R}^{N\times d},\end{aligned}$$

where softmax is applied row-wise and $\alpha = 1/\sqrt{d}$ is the scaling factor. In practice, we subtract rowmax($\mathbf{S}$) from $\mathbf{S}$ for numerical stability. For multi-head attention (MHA), each head has its own set of projections, and this computation parallelizes across multiple heads and batches.

Given output grad $\mathbf{dO} \in \mathbb{R}^{N\times d}$, the backward computes:

$$\begin{aligned}\mathbf{dV} &= \mathbf{P}^\top\mathbf{dO}, \quad \mathbf{dP} = \mathbf{dO}\mathbf{V}^\top,\\ \mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}),\\ \mathbf{dQ} &= \alpha\mathbf{dS}\mathbf{K}, \quad \mathbf{dK} = \alpha\mathbf{dS}^\top\mathbf{Q},\end{aligned}$$

where dsoftmax($\mathbf{dP}$) denotes row-wise softmax gradient $\mathbf{d}s = (\mathrm{diag}(p) - pp^\top)\mathbf{d}p$ for $p = \mathrm{softmax}(s)$.

2.2 GPU Hardware Characteristics and Execution Model

We describe the aspects of the GPU's execution model relevant for FlashAttention-4, with a focus on the NVIDIA Blackwell architecture (B200 & GB200). We highlight key differences from the prior Hopper architecture that motivate the optimizations in FlashAttention-4.

Memory hierarchy: The GPU's memories are organized as a hierarchy of data locales, with capacity inversely related to bandwidth. Global memory (GMEM), also known as HBM, is the off-chip DRAM that is accessible to all streaming multiprocessors (SMs). Data from GMEM are transparently cached in an on-chip L2 cache. Next, each SM contains a small, programmer-managed, highly banked cache called shared memory (SMEM) on the chip. Lastly, there is the register file within each SM.

Blackwell introduces a new memory level called *tensor memory* (TMEM), a 256 KB on-chip memory per SM specifically designed for storing intermediate results of tensor core operations. Unlike shared memory, TMEM is warp-synchronous and tightly coupled with the tensor cores, enabling the matrix multiply-accumulate (MMA) units to write outputs directly to TMEM without consuming registers. This alleviates the extreme register pressure that plagued Hopper kernels and

工作效率，降低了入门门槛，使得研究人员即便没有深厚的C++模板元编程知识，也能快速构建原型并部署新的注意力机制变体。

为对我们的方法进行实证验证，我们在B200 GPU上对 FlashAttention-4进行了基准测试，结果表明：(1) BF16格式相较于cuDNN可实现高达1.3× 的加速比，相较于Triton实现方案则可达2.7× 的加速比；(2) 我们能够在经过优化的瓶颈资源上实现接近极限的利用率，浮点运算次数最高可达 ∼1600 TFLOPS，这一数值相当于理论最大值的71%；(3) 在处理长序列长度的数据时， FlashAttention-4的性能优于其他现有的注意力机制实现方案。

我们以宽松的许可协议将 FlashAttention-4 开源，并致力于将其与流行的编程库整合，以便让最多的研究人员和开发者受益。相关代码可在 https://github.com/Dao-AILab/flash-attention/tree/main/flash_attn/cute地址获取。

2 背景

2.1 多头注意力机制

令$\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$ $\in \mathbb{R}^{N\times d}$ 分别表示与单个头相关的查询、键和值输入序列，其中N 为序列长度， d 为头维度。注意力输出$\mathbf{O} \in \mathbb{R}^{N\times d}$ 的计算公式为：

$$\begin{aligned}\mathbf{S} &= \alpha\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N\times N},\\ \mathbf{P} &= \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N\times N},\\ \mathbf{O} &= \mathbf{P}\mathbf{V} \in \mathbb{R}^{N\times d},\end{aligned}$$

其中softmax函数是按行应用的， $\alpha = 1/\sqrt{d}$ 则为缩放因子。在实际应用中，为保证数值稳定性，我们会从$\mathbf{S}$中减去rowmax($\mathbf{S}$)的值。在多头注意力（MHA）机制中，每个注意力头都拥有独立的投影矩阵，且该计算过程可在多个注意力头及批次数据之间并行执行。

给定输出梯度 $\mathbf{dO} \in \mathbb{R}^{N\times d}$，反向传播过程计算如下：

$$\begin{aligned}\mathbf{dV} &= \mathbf{P}^\top\mathbf{dO}, \quad \mathbf{dP} = \mathbf{dO}\mathbf{V}^\top,\\ \mathbf{dS} &= \mathrm{dsoftmax}(\mathbf{dP}),\\ \mathbf{dQ} &= \alpha\mathbf{dS}\mathbf{K}, \quad \mathbf{dK} = \alpha\mathbf{dS}^\top\mathbf{Q},\end{aligned}$$

其中 dsoftmax($\mathbf{d}$) 的值为 $\mathbf{P}$表示按行计算的softmax梯度，$\mathbf{d}s = (\mathrm{diag}(p) - pp^\top)\mathbf{d}p$ 即为 $p = \mathrm{softmax}(s)$的该梯度。.

2.2 GPU的硬件特性与执行模型

我们阐述了与 FlashAttention-4相关的GPU执行模型各要素，重点讨论了NVIDIA Blackwell架构（B200及GB200）。同时指出了该架构与之前的Hopper架构之间的关键差异，这些差异正是推动 FlashAttention-4中各项优化措施产生的原因。

内存层次结构：GPU的存储器被组织为一种由数据存储单元构成的层次结构，其容量与带宽呈反比关系。被称为HBM的全局内存（GMEM）实际上就是片外DRAM，所有流式多处理器（SM）均可访问它。来自全局内存的数据会自动被缓存在片上的L2缓存中。此外，每个SM内部还拥有一个由程序员管理、具有多个存储Bank的小型缓存，即片上共享内存（SMEM）。最后，每个SM内还设有寄存器文件。

Blackwell架构引入了一种名为张量内存（TMEM）的新层次内存，即每个流式多处理器所配备的256 KB片上内存，专为存储张量核运算的中间结果而设计。与共享内存不同，TMEM具备线程同步特性，并与张量核心紧密相连，从而使多精度算术单元能够直接将运算结果写入TMEM，无需占用寄存器。这一设计有效缓解了此前困扰Hopper架构内核的严重寄存器压力，

enables larger tile sizes. TMEM is allocated in 32-column (16 KB) granules and requires explicit programmer management for allocation, deallocation, and data movement.

Thread hierarchy: The GPU's programming model is organized around logical groupings of execution units called threads. From the finest to the coarsest level, the thread hierarchy is comprised of threads, warps (32 threads), warpgroups (4 contiguous warps), threadblocks (i.e. cooperative thread arrays or CTAs), threadblock clusters, and grids. Threads in the same CTA are co-scheduled on the same SM, and CTAs in the same cluster are co-scheduled on the same GPC. SMEM is directly addressable by all threads within a CTA, whereas each thread has at most 256 registers (RMEM) private to itself.

Tensor cores and increased asynchrony: Blackwell features fifth-generation tensor cores that operate on tiles significantly larger than in previous architectures. Each MMA tensor core instruction processes $128 \times N$ tiles (typically $N = 128$ or 256), compared to $64 \times N$ on Hopper. Crucially, Blackwell MMAs write their output directly to TMEM asynchronously, whereas Hopper MMAs write to registers. This full asynchrony enables better overlap between computation and other operations, as the MMA units no longer block on register writeback.

Hardware support for asynchrony allows for warp-specialized kernels, where the warps of a CTA are divided into producer or consumer roles that only ever issue data movement or computation [1].

2-CTA tensor core: Blackwell supports a 2-CTA tensor core MMA mode in which a CTA pair within the same thread block cluster cooperatively executes a single MMA, allowing the operation to read and write tensor memory from both CTAs. One thread in the pair initiates the MMA, but the peer CTA must be launched and remain active while the operation is in flight. Compared to single CTA MMAs that limit the dimension M to 128, the paired mode supports M = 128 or 256 by partitioning the A tile and the accumulator across the pair in dimension M and partitioning the B tile across the two CTAs in dimension N so that each CTA stages only half of B in its own shared memory while the hardware consumes the combined B tile during the multiply. This reduces redundant shared memory capacity and bandwidth, but since these operations touch tensor memory across the CTA pair, kernels must launch CTAs in fixed pairs and use a consistent 2-CTA mode for tensor memory and tensor core operations throughout the kernel.

Shifting bottlenecks: A key trend reflected in Blackwell is that tensor core throughput scales faster than other functional units. Blackwell doubles the FP16/BF16 tensor core throughput compared to Hopper (2.25 PFLOPS [19] vs 1 PFLOPS [17] per GPU), but shared memory bandwidth and exponential unit throughput remain unchanged or scale more slowly. This imbalance shifts the performance bottleneck away from matrix multiplication toward shared memory traffic and non-matmul operations like softmax. As our roofline analysis in Sections 3.1 and 3.2 shows, this requires careful kernel design to maximize overlap between MMA operations and these bottleneck resources.

The throughput of several hardware components on B200 (and GB200) is listed below.

1. Tensor cores: The BF16 MMA has a throughput of 8192 ops / clock / SM, doubled from 4096 ops / clock / SM of Hopper. This can be derived from the theoretical maximum FLOPS: 2.25 PFLOPS / 1850 Mhz clock speed / 148 SMs = 8192 ops / clock / SM.
2. Exponential unit. The multifunction unit (MUFU) on B200 and GB200 can perform 16 ops / clock / SM, the same as Hopper [18]. We note that B300 and GB300 GPUs have doubled the exponential throughput to 32 ops / clock / SM, though these GPUs are not yet widely available at the time of writing.
3. SMEM: the read throughput is 128 bytes / clock / SM, the same as Hopper, as measured by microbenchmarking [15].

进而使得更大的数据块尺寸成为可能。TMEM以32列（16 KB）为单位进行分配，程序员需要手动负责其分配、释放以及数据传输等相关操作。

线程层次结构：GPU的编程模型是以被称为线程的执行单元逻辑分组为基础构建的。从最细粒度到最粗粒度，线程层次结构依次由线程、线程组（32个线程）、线程组群（4个连续的线程组）、线程块（即协作线程数组或CTA）、线程块集群以及网格构成。同一CTA中的线程会在同一个SM上被共同调度，而同一集群中的CTA则会在同一个GPC上被共同调度。CTA内的所有线程均可直接访问SMEM，而每个线程最多拥有256个属于自己的私有寄存器（RMEM）。

张量核心与更高的异步性：Blackwell架构配备了第五代张量核心，其处理单元的尺寸远大于前代架构。每个MMA张量核心指令可处理 $128 \times N$ 个处理单元块（通常为 $N = 128$ 或256个），而Hopper架构仅为 $64 \times N$ 个。更为关键的是，Blackwell架构中的MMA单元会以异步方式将计算结果直接写入TMEM，而Hopper架构的MMA单元则需先将结果写入寄存器。这种完全的异步处理机制使得计算操作与其他操作之间能实现更好的并行执行，因为MMA单元不再会因寄存器写回操作而受阻。

对异步操作的硬件支持使得专门为线程组设计的内核成为可能，在这类内核中，CTA中的各个线程组会被划分为生产者或消费者角色，它们仅负责发起数据传输或计算操作 [1]。

2-CTA tensor core: Blackwell supports a 2-CTA tensor core MMA mode in which a CTA pair within the same thread block cluster cooperatively executes a single MMA, allowing the operation to read and write tensor memory from both CTAs. One thread in the pair initiates the MMA, but the peer CTA must be launched and remain active while the operation is in flight. Compared to single CTA MMAs that limit the dimension M to 128, the paired mode supports M = 128 or 256 by partitioning the A tile and the accumulator across the pair in dimension M and partitioning the B tile across the two CTAs in dimension N so that each CTA stages only half of B in its own shared memory while the hardware consumes the combined B tile during the multiply. This reduces redundant shared memory capacity and bandwidth, but since these operations touch tensor memory across the CTA pair, kernels must launch CTAs in fixed pairs and use a consistent 2-CTA mode for tensor memory and tensor core operations throughout the kernel.

瓶颈的转移：Blackwell架构中一个显著的趋势是，张量核心的吞吐量增长速度快于其他功能单元。与Hopper架构相比，Blackwell架构下FP16/BF16格式的张量核心吞吐量提升了一倍（每颗GPU的吞吐量分别为2.25千万次浮点运算每秒 [19] 和1千万次浮点运算每秒 [17] ），而共享内存的带宽以及指数运算单元的吞吐量则保持不变或增长更为缓慢。这种失衡使得性能瓶颈从矩阵乘法转向了共享内存流量以及softmax函数等非矩阵乘法操作。正如我们在3.1节和3.2节所做的性能边界分析所示，为了解决这一问题，必须通过精心设计内核，尽可能提高多精度算术单元操作与这些瓶颈资源之间的重叠效率。

The throughput of several hardware components on B200 (and GB200) is listed below.

1. Tensor cores: The BF16 MMA has a throughput of 8192 ops / clock / SM, doubled from 4096 ops / clock / SM of Hopper. This can be derived from the theoretical maximum FLOPS: 2.25 PFLOPS / 1850 Mhz clock speed / 148 SMs = 8192 ops / clock / SM.
2. Exponential unit. The multifunction unit (MUFU) on B200 and GB200 can perform 16 ops / clock / SM, the same as Hopper [18]. We note that B300 and GB300 GPUs have doubled the exponential throughput to 32 ops / clock / SM, though these GPUs are not yet widely available at the time of writing.
3. SMEM: the read throughput is 128 bytes / clock / SM, the same as Hopper, as measured by microbenchmarking [15].

We see that the MMA throughput on Blackwell has doubled compared to Hopper, but other hardware units do not necessarily get faster at the same rate. This reflects a broader trend in accelerator design: increasing the throughput of the most important components (typically matrix multiply units) to get higher performance under similar power / silicon area constraint.

3 Algorithm

3.1 Attention forward pass

We first do a roofline analysis to show the bottlenecks of attention forward pass, which motivates our new pipeline design, as well as changes in the FlashAttention algorithm to increase the throughput of the exponential unit and avoid most of the softmax rescaling steps.

3.1.1 Feeds and Speeds

We provide intuition for our kernel design and optimizations by first analyzing the roofline, based on the throughput of the matmul units (tensor cores), shared memory (smem), and exponential unit. We note that this is a simplified analysis that does not consider all resources in the GPU (e.g., floating point math, register bandwidth, L2 bandwidth). Nevertheless, it can identify bottlenecks.

Let the shape of the tile along the length dimension of the sequence of $\mathbf{Q}$ and $\mathbf{K}$ be $M \times N$, and let the head dimension be d. We analyze the compute and memory traffic requirements to identify the performance bottleneck.

MMA compute. The forward pass performs two matrix multiply-accumulate (MMA) operations per iteration: $\mathbf{QK}^\top$ (computing $M \times N$ output from $M \times d$ and $d \times N$ inputs) and $\mathbf{PV}$ (computing $M \times d$ output from $M \times N$ and $N \times d$ inputs). Each MMA requires $2MNd$ floating-point operations. With a tensor core throughput of 8192 FLOPs per cycle, the total compute time is

$$T_{\text{MMA}} = \frac{4MNd}{8192} \text{ cycles.} \tag{1}$$

Shared memory traffic. Of the two MMAs, one is shared-shared (SS) where both operands are read from shared memory ($\mathbf{QK}^\top$), while the other is tensor-shared (TS) where operand A is read from tensor memory and operand B from shared memory ($\mathbf{PV}$). Since each MMA instruction operates on tiles of size 128×128, computing an $M \times N$ output requires $\lceil M/128 \rceil \times \lceil N/128 \rceil$ MMA instructions. Crucially, when multiple MMA instructions are needed, the shared memory operands are read multiple times.

For $\mathbf{QK}^\top$ (SS), computing the $M \times N$ output requires $\lceil M/128 \rceil \times \lceil N/128 \rceil$ MMA instructions, each reading a $128 \times d$ chunk of $\mathbf{Q}$ and a $d \times 128$ chunk of $\mathbf{K}^\top$ from shared memory. The total shared memory reads are $\lceil M/128 \rceil \times \lceil N/128 \rceil \times (128d + 128d) = \lceil M/128 \rceil \lceil N/128 \rceil \times 256d$ elements. For $\mathbf{PV}$ (TS), computing the $M \times d$ output requires $\lceil M/128 \rceil \times \lceil d/128 \rceil$ MMA instructions, each reading a $N \times 128$ chunk of $\mathbf{V}$ from shared memory, totaling $\lceil M/128 \rceil \times \lceil d/128 \rceil \times 128N$ elements. At 2 bytes per element (bf16) and 128 bytes per cycle bandwidth, the shared memory (T_{smem}) read time is

$$= 2\left\lceil \tfrac{M}{128} \right\rceil \left\lceil \tfrac{N}{128} \right\rceil 256d + 2\left\lceil \tfrac{M}{128} \right\rceil \left\lceil \tfrac{d}{128} \right\rceil 128N = \tfrac{3MNd}{8192} \text{ cycles} \tag{2}$$

(assuming M, N, d are multiples of 128).

Exponential unit. The exponential unit computes elementwise operations required for the softmax computation. The forward pass requires exponential operations on $M \times N$ values (corresponding to the attention matrix $\mathbf{S}$). With a throughput of 16 operations per cycle, the exponential unit requires

$$T_{\text{exp}} = \frac{MN}{16} \text{ cycles.} \tag{3}$$

Table 1 summarizes the analysis for two typical tile configurations. For $M = N = d = 128$, the resources are well-balanced, with shared memory (768 cycles) being slightly lower than both MMA

我们发现，在Blackwell架构上，MMA的吞吐量相较于Hopper架构提升了一倍，但其他硬件单元的加速幅度并不一定与之相同。这反映了加速器设计中的一个更普遍的趋势：即在相同的功耗/芯片面积限制下，通过提升最重要组件（通常是矩阵乘法单元）的吞吐量来实现更高的性能。

3 算法

3.1 注意力机制的前向传播

我们首先进行性能边界分析，以找出注意力前向传播中的瓶颈，这一发现为我们的新处理流程设计提供了依据，同时也促使我们改进 FlashAttention 算法，从而提升指数运算单元的吞吐量，并减少大部分softmax函数的重新缩放步骤。

3.1.1 Feeds和Speeds

我们首先基于矩阵乘法单元（张量核心）、共享内存（SMEM）以及指数运算单元的吞吐量来分析系统的轮廓线，从而为我们的内核设计及优化措施提供直观的依据。需要说明的是，这是一种简化的分析方法，并未考虑GPU中的所有资源（如浮点运算功能、寄存器带宽、L2缓存带宽等），但它依然能够帮助我们找出性能瓶颈。

设$\mathbf{Q}$与$\mathbf{K}$序列在长度维度上的块状结构为 $M \times N$，头维度则为 d。我们通过分析计算与内存流量的需求，以此定位性能瓶颈。

MMA运算。在前向传播过程中，每次迭代会执行两次矩阵乘累加操作：$\mathbf{QK}^\top$（根据 $M \times d$和 $d \times N$输入计算 $M \times N$输出），以及$\mathbf{PV}$（根据 $M \times N$和 $N \times d$输入计算$M \times d$输出）。每次MMA运算都需要 $2MNd$次浮点运算。由于张量核心的每周期吞吐量为8192次浮点运算次数，因此总的运算时间为

$$T_{\text{MMA}} = \frac{4MNd}{8192} \text{ cycles.} \tag{1}$$

共享内存流量。在两个多精度算术单元中，有一种为共享-共享型（SS），其两个操作数均从共享内存（$\mathbf{QK}^\top$）中读取；另一种为张量-共享型（TS），其中操作数 A从张量内存中读取，而操作数 B 则从共享内存（$\mathbf{PV}$）中读取。由于每个多精度算术单元指令都是对大小为 128×128的块进行运算，因此生成一个 $M \times N$输出需要 $\lceil M/128 \rceil \times \lceil N/128 \rceil$ 个多精度算术单元指令。至关重要的是，当需要多个多精度算术单元指令时，共享内存中的操作数会被多次读取。

对于$\mathbf{QK}^\top$ (SS)，计算 $M \times N$输出需要 $\lceil M/128 \rceil \times \lceil N/128 \rceil$ 条MMA指令，每条指令都会从共享内存中读取一块 $128 \times d$ 大小的$\mathbf{Q}$ 数据以及一块 $d \times 128$ 大小的$\mathbf{K}^\top$ 数据。因此总共需要读取 $\lceil M/128 \rceil \times \lceil N/128 \rceil \times (128d + 128d) = \lceil M/128 \rceil \lceil N/128 \rceil \times 256d$ 个元素的数据到共享内存中。而对于 $\mathbf{PV}$ (TS)，计算 $M \times d$ 输出同样需要 $\lceil M/128 \rceil \times \lceil d/128 \rceil$ 条MMA指令，每条指令会从共享内存中读取一块 $N \times 128$ 大小的$\mathbf{V}$ 数据，总计需要读取 $\lceil M/128 \rceil \times \lceil d/128 \rceil \times 128N$ 个元素。在每个元素占用2字节（BF16格式）且每个周期的带宽为128字节的情况下，共享内存(T_{smem})的读取时间即为

$$= 2\left\lceil \tfrac{M}{128} \right\rceil \left\lceil \tfrac{N}{128} \right\rceil 256d + 2\left\lceil \tfrac{M}{128} \right\rceil \left\lceil \tfrac{d}{128} \right\rceil 128N = \tfrac{3MNd}{8192} \text{ cycles} \tag{2}$$

（假设 M、 N、 d均为multiples of 128）

指数运算单元。该指数运算单元负责执行softmax函数计算所需的逐元素运算。在前向传播过程中，需要对 $M \times N$ 个数值（即注意力矩阵$\mathbf{S}$中的元素）进行指数运算。由于该单元每周期可完成16次运算，因此需要……

$$T_{\text{exp}} = \frac{MN}{16} \text{ cycles.} \tag{3}$$

表1汇总了两种典型瓦片配置的分析结果。对于 $M = N = d = 128$而言，各资源的分配较为均衡，其中共享内存的循环次数为768次，略低于MMA运算及指数运算单元的循环次数（均为1024次）。

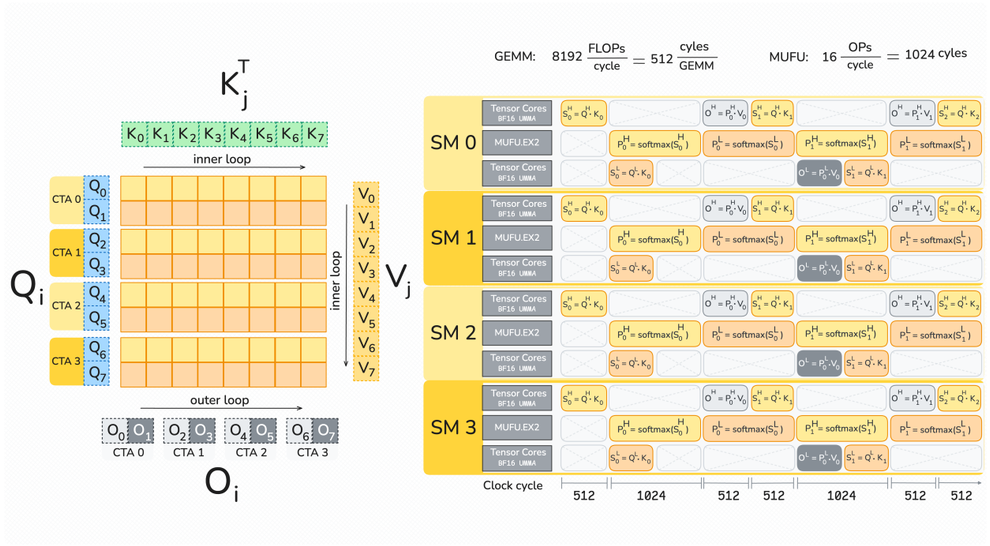


Figure 1: FlashAttention-4 forward pipeline. The superscript ^{H} denotes the matrices corresponding to the "high" Q tile, and superscript ^{L} denotes matrices corresponding to the "low" Q tile. Each Q tile corresponds to 128 query tokens.

Resource	128^3	256×128^2
MMA compute	**1024**	**2048**
Shared memory	768	1536
Exponential unit	**1024**	**2048**

Table 1: Roofline analysis (cycles) for the attention forward pass. For both tile sizes, MMA compute and exponential unit are the primary bottlenecks.

compute and exponential unit (both 1024 cycles). For the larger tile size $M = 256, N = d = 128$, the shared memory traffic increases to 1536 cycles due to reading MMA operands multiple times, while MMA compute and exponential unit double to 2048 cycles. This analysis motivates our kernel design to (1) use large tile sizes and maximize overlap between MMA operations and softmax computations (2) increase the throughput of exponential by using other hardware units (3) reduce the time of unnecessary non-matmul operations.

3.1.2 New pipeline to overlap matmul and softmax

Since the Blackwell architecture doubled the tensor core flops again, taking care to overlap softmax and tensor core operations is even more crucial than on Hopper. We follow a ping-pong schedule similar to FA-3, where two tiles of the output are computed per thread block. While one tile's tensor core operations are executed, the other tile computes softmax. While Hopper tensor cores hold the accumulator in registers, with four threads per row in an interleaved pattern, Blackwell tensor cores hold their accumulators in tensor memory. Additionally, a single accumulator tile on Blackwell is 128 by 128 elements large, where Hopper's tile size was 64 by 128.

The natural way to distribute work across these tiles is then to have two warpgroups of 128

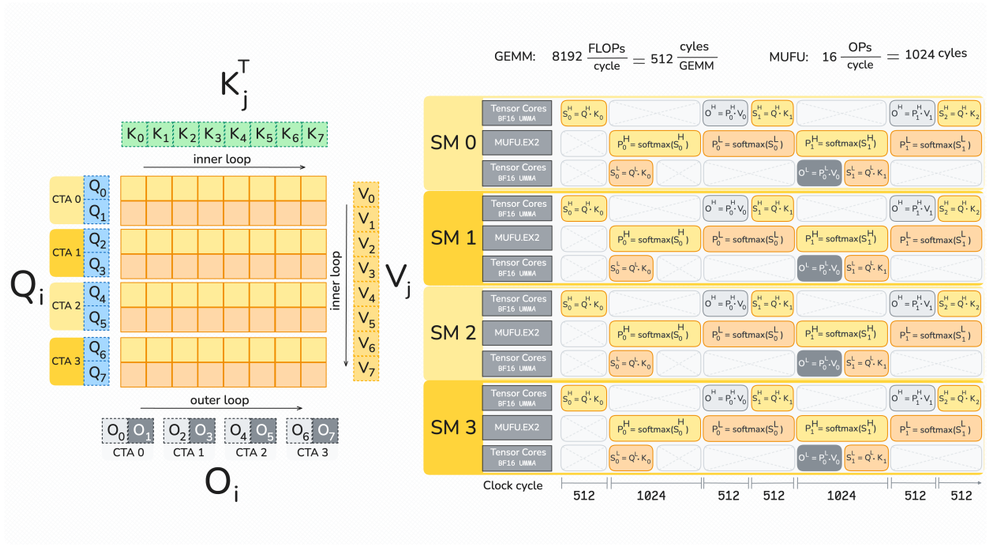


图1：FlashAttention-4的前向传播流程。上标 ^{H} 表示对应于“高”Q块的矩阵，而上标 ^{L} 则表示对应于“低”Q块的矩阵。每个Q块对应128个查询标记。

资源	128^3	256×128^2
MMA运算	**1024**	**2048**
共享内存	768	1536
指数运算单元	**1024**	**2048**

表1：注意力前向传播的性能边界分析（以循环次数表示）。对于这两种Q块尺寸而言，MMA运算与指数运算单元都是主要的瓶颈所在。

当瓦片尺寸增大至 $M = 256$那样 $N = d = 128$时，由于需要多次读取MMA运算的操作数，共享内存的流量上升至1536次循环；与此同时，MMA运算及指数运算单元的循环次数也增至2048次。基于这些分析，我们的内核设计采取了以下策略：(1)采用较大的瓦片尺寸，尽可能提升MMA运算与softmax函数计算之间的重叠度；(2)利用其他硬件单元来提高指数运算的处理效率；(3)减少那些不必要的非矩阵乘法运算所占用的时间。

3.1.2 用于实现matmul与softmax操作重叠的新流水线

由于Blackwell架构再次使张量核心的浮点运算次数翻倍，因此相较于Hopper架构，更加注重让softmax函数与张量核运算实现重叠变得至关重要。我们采用了与FA-3类似的乒乓式调度策略，即每个线程块负责计算输出中的两个数据块。当一个数据块的张量核运算正在执行时，另一个数据块则负责计算softmax函数。Hopper架构下的张量核心会将累加器存储在寄存器中，且每行以交错方式排列四个线程，而Blackwell架构下的张量核心则将累加器存储在张量内存中。此外，Blackwell架构中单个累加器数据块的规模为128×128个元素，而Hopper架构的该数值则为64×128。

因此，要在这些分块之间自然地分配任务，最佳方式就是设置两个各包含128个元素的线程组群。

threads each, with each thread processing an entire row. This eliminates the need for inter-warp shuffles to reduce the row max, and for multiple statistics registers per thread. Just like with FA-3, we explicitly synchronize the two softmax warpgroups to not overlap in their critical section, which is the part of exponential computation. Each softmax warpgroup proceeds by first loading the entire row into registers, then computing the maximum, then computing the softmax (i.e., subtract the max, rescale, exponentiate, convert to input precision), and finally computing the row sum.

Another difference from FA-3 is that since we transfer **P** via tensor memory rather than register file, we can decouple the rescaling of the output to a separate "correction" warpgroup and thus take it out of the critical path.

Several tensor memory partitionings are possible to achieve this pipeline overlap. All must allocate two tiles worth of output, leaving (at head dimension 128) half the tensor memory to store **S** and **P**. That memory can store two copies of **S** or four copies of **P** (assuming the input of the FP16 or BF16 tensor core). This leaves us with roughly two partitioning options for the remaining tensor memory: one tile of **S** and two tiles of **P**, or two tiles of **S** that overlap with **P**. We choose the latter because it allows us to start our software pipeline by immediately computing two **S** tiles. It also leaves some tensor memory to communicate rescale statistics to the correction warpgroup.

One issue of the larger Blackwell tile sizes and the chosen thread assignment is that, unless we re-load from tensor memory, we must hold an entire row of 128 elements in register. Given that we use two softmax warpgroups, one correction warpgroup, and one warpgroup to drive tensor cores and TMA units, assigning sufficient registers to softmax and preventing register spills is critical. For BF16 input data types, we need to hold 128 registers for the input, and potentially 64 registers for the output (plus miscellaneous and temporary registers). To reduce register pressure, we stage out storing **P**: The first three quarters are stored once (and trigger the corresponding MMA operations), and the last quarter is stored separately.

3.1.3 Emulation of the exponential function

Exponential throughput bottleneck. On modern GPUs, the exponential function is computed by the multi-function unit (MUFU), which has significantly lower throughput than the tensor cores used for matrix multiplication. On B200 and GB200 GPUs, MUFU provides 16 operations per clock per SM, compared to 8192 operations per clock per SM for matrix multiplication. Since softmax computation requires many exponential evaluations, this disparity makes the exponential function a critical bottleneck in attention kernels.

Software emulation via polynomial approximation. To increase exponential throughput, we implement a software emulation of 2^x using floating-point FMA units, which can operate in parallel with MUFU. We use the classical range reduction technique (Cody-Waite) and then the polynomial approximation [16]. The key insight is to decompose the exponential computation:

$$2^x = 2^{\lfloor x \rfloor} \, 2^{x - \lfloor x \rfloor} \tag{4}$$

where $\lfloor x \rfloor$ is the integer part and $x - \lfloor x \rfloor \in [0, 1)$ is the fractional part.

The integer part $2^{\lfloor x \rfloor}$ can be computed efficiently using bit manipulation of the IEEE 754 floating-point representation. Since the exponent field directly represents powers of two, computing $2^{\lfloor x \rfloor}$ amounts to a shift and add operation on the exponent bits, which can be done using integer ALU instructions.

For the fractional part, we approximate $2^{x_{\text{frac}}}$ where $x_{\text{frac}} \in [0, 1)$ using a polynomial:

$$2^{x_{\text{frac}}} \approx \sum_{i=0}^{n} p_i \, x_{\text{frac}}^i \tag{5}$$

with $p_0 = 1.0$ and the remaining coefficients chosen to minimize the relative approximation error

与FA-3相同，我们也会对这两个用于计算softmax函数的线程组群进行同步处理，避免它们在处理指数运算这一核心环节时出现时间冲突。每个线程组群的操作流程为：首先将整行数据加载到寄存器中，然后找出该行的最大值，接着计算softmax函数值——具体步骤包括从最大值中减去对应数值、重新缩放、取指数，最后将结果转换回与输入数据相同的精度。最后，再计算出该行的总和。

与FA-3的另一个不同之处在于，由于我们是通过张量内存而非寄存器文件来传输<span style id='1'>P值，因此可以将输出的重缩放操作分离到单独的“校正”线程组中，从而将其移出关键路径。

Several tensor memory partitionings are possible to achieve this pipeline overlap. All must allocate two tiles worth of output, leaving (at head dimension 128) half the tensor memory to store **S** and **P**. That memory can store two copies of **S** or four copies of **P** (assuming the input of the FP16 or BF16 tensor core). This leaves us with roughly two partitioning options for the remaining tensor memory: one tile of **S** and two tiles of **P**, or two tiles of **S** that overlap with **P**. We choose the latter because it allows us to start our software pipeline by immediately computing two **S** tiles. It also leaves some tensor memory to communicate rescale statistics to the correction warpgroup.

较大的Blackwell瓦片尺寸以及所选的线程分配方式带来一个问题：除非从张量内存中重新加载数据，否则我们必须将包含128个元素的整行数据存储在寄存器中。鉴于我们使用了两个用于执行softmax运算的线程组群、一个用于校正计算的线程组，以及一个用于驱动张量核心和TMA单元的线程组，因此为softmax运算分配足够的寄存器并避免寄存器溢出显得至关重要。对于BF16格式的输入数据，我们需要为输入数据预留128个寄存器，输出数据则可能需要64个寄存器（此外还需考虑各类其他及临时寄存器）。为了减轻寄存器压力，我们采用了分阶段存储**P**值的方式：前四分之三的数值会一次性存储（并触发相应的多精度算术单元运算），而最后的四分之一则会单独存储。

3.1.3 指数函数的仿真

指数级吞吐量瓶颈。在现代GPU上，指数函数是由多功能单元（MUFU）来计算的，其吞吐量远低于用于矩阵乘法的张量核心。在B200和GB200 GPU上，每个流式多处理器（SM）的MUFU每时钟周期可执行16次操作，而矩阵乘法则可达每时钟周期每SM 8192次操作。由于softmax计算需要大量指数函数运算，这种差距使得指数函数成为注意力内核中的关键瓶颈。

通过多项式近似实现软件仿真。为提升指数级处理效率，我们采用了基于浮点FMA单元的 2^x 软件仿真方案，这类单元能够与多功能单元并行工作。我们首先运用经典的数值范围压缩技术（Cody-Waite算法），然后再进行多项式近似处理 [16]。其核心思路在于将指数运算分解开来：

$$2^x = 2^{\lfloor x \rfloor} \, 2^{x - \lfloor x \rfloor} \tag{4}$$

其中 $\lfloor x \rfloor$ 为整数部分， $x - \lfloor x \rfloor \in [0, 1$为小数部分。

利用IEEE 754浮点数表示格式的位操作，可以高效地计算整数部分 $2^{\lfloor x \rfloor}$ 。由于指数字段直接表示2的幂次，因此计算$2^{\lfloor x \rfloor}$ 实际上只需对指数位进行移位和加法运算，这类操作可通过整数ALU指令来完成。

对于小数部分，我们采用多项式来近似 $2^{x_{\text{frac}}}$ （其中 $x_{\text{frac}} \in [0, 1)$ ，即：

$$2^{x_{\text{frac}}} \approx \sum_{i=0}^{n} p_i \, x_{\text{frac}}^i \tag{5}$$

包含 $p_0 = 1.0$ 以及 其余的系数则是为最小化相对近似误差而选定的。 r

Method	FP32 vs FP64		BF16 vs FP64	
	Max rel err	Mean rel err	Max rel err	Mean rel err
Ideal (FP64→BF16)	—	—	3.89×10^{-3}	1.41×10^{-3}
Hardware `MUFU.EX2`	1.41×10^{-7}	3.04×10^{-8}	3.89×10^{-3}	1.41×10^{-3}
Degree 3	8.77×10^{-5}	5.43×10^{-5}	3.90×10^{-3}	1.41×10^{-3}
Degree 4	3.05×10^{-6}	1.84×10^{-6}	3.89×10^{-3}	1.41×10^{-3}
Degree 5	1.44×10^{-7}	5.48×10^{-8}	3.89×10^{-3}	1.41×10^{-3}

Table 2: Accuracy of 2^x polynomial emulation on $[0, 1)$, measured against FP64 reference on 4M random inputs. FP32 columns measure the raw polynomial output; BF16 columns measure after rounding to BF16. The BF16 quantization error dominates for all degrees ≥ 3.

over $[0, 1)$, calculated using the Sollya software package [4]. The polynomial evaluation uses Horner's method with FMA instructions, achieving high throughput.

The complete algorithm proceeds as follows:

1. Clamp x to be at least -127 to avoid underflow
2. Compute $\lfloor x \rfloor$ using round-down mode: add $2^{23} + 2^{22}$ to x (forcing the fractional bits into the mantissa), then subtract it back with round-down mode
3. Compute fractional part: $x_{\text{frac}} = x - \lfloor x \rfloor$
4. Evaluate polynomial to get $2^{x_{\text{frac}}}$
5. Combine integer and fractional parts: shift $\lfloor x \rfloor$ into the exponent field and add the mantissa bits of $2^{x_{\text{frac}}}$

By distributing exponential computations across both MUFU and FMA units, this approach effectively increases the exponential throughput, alleviating a key bottleneck in attention computation.

Partial emulation. Although polynomial emulation increases exponential throughput, it comes at a cost: additional registers (to hold intermediate values and coefficients), higher register bandwidth consumption, and longer latency compared to the MUFU instruction. Using emulation for all exponential evaluations would increase register pressure and could cause spills that negate the throughput benefit. Instead, we apply emulation to only a subset of the entries in each softmax row (10–25%), with the remaining entries computed via hardware `MUFU.EX2`. The exact fraction is tuned empirically based on the ratio of MMA and exponential throughput for a given tile configuration.

Numerical accuracy. Table 2 compares the accuracy of polynomial approximations of different degrees against the hardware `MUFU.EX2` instruction, measured on 4M random inputs in $[0, 1)$. We report two metrics: the FP32-level error (before any quantization) and the BF16-level error (after rounding the FP32 output to BF16), both measured against a FP64 reference.

At the FP32 level, the degree-3 polynomial has a maximum relative error of 8.8×10^{-5}, roughly $600\times$ higher than hardware. However, after rounding to BF16, the errors become nearly indistinguishable: the quantization error of BF16 ($\sim 3.9 \times 10^{-3}$) dominates the polynomial approximation error for all degrees ≥ 3. The degree-3 polynomial matches hardware to within 1 BF16 ULP on 99% of inputs, which is sufficient for attention computation where the softmax output is consumed with BF16 precision. Higher-degree polynomials close the FP32 gap: degree 5 matches hardware to within $2\times$ in maximum relative error, at the cost of two additional FMA instructions per evaluation.

研究方法	FP32格式与FP64格式的对比		BF16格式与FP64格式的对比	
	最大相对误差	平均相对误差	最大相对误差	平均相对误差
理想情况（FP64→BF16格式）	—	—	3.89×10^{-3}	1.41×10^{-3}
Hardware MUFU.EX2硬件	1.41×10^{-7}	3.04×10^{-8}	3.89×10^{-3}	1.41×10^{-3}
三次多项式	8.77×10^{-5}	5.43×10^{-5}	3.90×10^{-3}	1.41×10^{-3}
四次多项式	3.05×10^{-6}	1.84×10^{-6}	3.89×10^{-3}	1.41×10^{-3}
五次多项式	1.44×10^{-7}	5.48×10^{-8}	3.89×10^{-3}	1.41×10^{-3}

表2：在 $[0, 1$上对 2^x 多项式进行仿真的精确度，该精度是针对400万个随机输入相对于FP64参考值来测量的。FP32格式的列显示的是多项式的原始输出结果，而BF16格式的列则显示经过BF16格式四舍五入后的结果。对于所有阶数 ≥ 3而言，BF16格式的量化误差都占据主导地位。

在 $[0, 1)$之上，该值是通过Sollya软件包 [4]计算得出的。多项式求值采用带有FMA指令的霍纳方法，从而实现高吞吐量。

整个算法的执行流程如下：1. 将 x 限制在至少 -127 的值，以避免下溢现象；2. 使用向下取整模式计算 $\lfloor x \rfloor$：先将 $2^{23} + 2^{22}$ 加到 x 上（从而将小数位强制纳入尾数部分），然后再以向下取整模式将其减回；3. 计算小数部分：$x_{\text{frac}} = x - \lfloor x \rfloor$；4. 通过多项式求值得到 $2^{x_{\text{frac}}}$。

5. 合并整数部分与小数部分：将 $\lfloor x \rfloor$ 移位到指数字段中，然后加上 $2^{x_{\text{frac}}}$的尾数位。

通过将指数运算分布在多功能单元与FMA单元之上，该方法有效提升了指数运算的吞吐量，从而缓解了注意力计算过程中的一个关键瓶颈。

部分仿真。尽管多项式仿真能够提升指数函数的运算吞吐量，但这也带来了相应的代价：需要更多的寄存器来存储中间值和系数，寄存器的带宽消耗更高，且与使用MUFU指令相比延迟也会更长。如果对所有的指数函数计算都采用仿真方式，将会加剧寄存器压力，甚至可能引发数据溢出，从而抵消掉吞吐量提升带来的好处。因此，我们仅对每行softmax中的部分元素（10–25%）应用仿真，其余元素则通过Hardware MUFU.EX2硬件直接计算。具体的比例会根据特定网格配置下多精度算术单元的运算能力与指数函数运算吞吐量的比值，通过实验进行优化调整。

数值精度。表2对比了在不同特征维度下，多项式近似方法相对于Hardware MUFU.EX2硬件的精确度，这些测试是在 $[0, 1$中包含400万条随机输入的数据上进行的。我们采用了两项指标来衡量精度：一项是FP32格式下的误差（在进行任何量化处理之前），另一项则是BF16格式下的误差（在将FP32格式的输出四舍五入为BF16格式之后），这两项指标都是以FP64格式作为参考标准来计算的。

在FP32格式下，3次多项式的最大相对误差为 8.8×10^{-5}，比硬件实现的误差大约高出600×。然而在转换为BF16格式后，这两种误差几乎无法区分：对于所有次数而言，BF16格式的量化误差($\sim 3.9 \times 10^{-3}$)都占据了主导地位，超过了多项式近似误差。在99%的输入数据上，3次多项式的计算结果与硬件实现的误差控制在1个BF16 ULP以内，这一精度已足以满足那些需要以BF16精度处理softmax输出的任务。更高次数的多项式则能进一步缩小与FP32格式的差距：5次多项式的最大相对误差可控制在 $2\times$ 以内，但为此每次计算需要多执行两条FMA指令。

3.1.4 Skipping online softmax rescaling

FlashAttention online softmax. FlashAttention computes attention $\text{softmax}(QK^\top)V$ in blocks to minimize memory traffic. For numerical stability, the algorithm maintains running statistics as it processes blocks. When computing block j, let $S_j = QK_j^\top$ be the attention scores for that block. The online softmax algorithm tracks:

$$m_j = \max(m_{j-1}, \text{rowmax}(S_j))$$
$$\ell_j = e^{m_{j-1}-m_j}\ell_{j-1} + \text{rowsum}(e^{S_j - m_j})$$

where m_j is the running max and ℓ_j is the running sum of exponentials (normalizer). The intermediate output O_j is updated as: $O_j = e^{m_{j-1}-m_j}O_{j-1} + e^{S_j-m_j}V_j$. The rescaling factor $e^{m_{j-1}-m_j}$ ensures numerical stability by renormalizing previous results when larger values are encountered.

Conditional rescaling. The step $e^{m_{j-1}-m_j}O_{j-1}$ requires a vector multiplication. We make two simple observations:

1. Rescaling is only necessary when $m_j > m_{j-1}$, i.e., when new larger values are found.
2. We can tolerate some "slack" in the rescaling: only rescale when $m_j - m_{j-1} > \tau$, where τ is a threshold (typically set to $\log_2(256) = 8.0$, corresponding to a rescaling factor of 256.0). As long as we keep track of the statistics (the total scaling we have done), we can still get the true denominator at the end to get the right final output.

In FlashAttention-4, we modify the algorithm as:

$$O_j = \begin{cases} e^{m_{j-1}-m_j}O_{j-1} + e^{S_j-m_j}V_j & \text{if } m_j - m_{j-1} > \tau \\ O_{j-1} + e^{S_j-m_{j-1}}V_j & \text{otherwise} \end{cases} \tag{6}$$

When $m_j - m_{j-1} \le \tau$, we skip updating m and continue using m_{j-1}. This maintains the correctness because at the end of the computation, all accumulated values are renormalized by the true maximum m_{final} and the final normalizer ℓ_{final}:

$$\text{Output} = \frac{1}{\ell_{\text{final}}}O_{\text{final}}$$

This modification significantly reduces the number of rescaling operations while maintaining numerical accuracy, as the final normalization step corrects any small deviations introduced by skipping intermediate rescaling.

In practice, to avoid warp divergence, we rescale when any of the threads in the warp needs rescaling.

3.2 Attention backward pass

3.2.1 Feeds and Speeds

Similar to the forward pass, we first provide intuition for our kernel design and optimizations by analyzing the roofline, based on the throughput of the matmul units (tensor cores), shared memory (smem), and exponential unit.

Let the tile shape along the sequence length dimension of **Q** and **K** be $M \times N$, and let the head dimension be d. We analyze the compute and memory traffic requirements to identify the performance bottleneck. Unlike for the forward pass, we make the assumption that $M = N = d = 128$ to simplify the formulas for the smem cycle count, although we retain variable names for clarity.

MMA compute. The backward pass performs five matrix multiply-accumulate (MMA) operations per iteration. Each MMA involves an $M \times N$ matrix, an $M \times d$ matrix, and an $d \times N$ matrix (varying which one is the output matrix), requiring $2MNd$ floating-point operations. With a tensor

3.1.4 跳过onlinesoftmax重缩放操作

FlashAttention算法的在线软最大化功能。FlashAttention通过分块计算注意力softmax($QK^\top$)V，以此降低内存流量。为保障数值稳定性，该算法在处理各数据块时会持续维护运行统计量。在计算第 j个数据块时，设 $S_j = QK_j^\top$为该块的注意力分数。在线软最大化算法会追踪这些数值。

$$m_j = \max(m_{j-1}, \text{rowmax}(S_j))$$
$$\ell_j = e^{m_{j-1}-m_j}\ell_{j-1} + \text{rowsum}(e^{S_j - m_j})$$

其中m_j 为指数值的滚动最大值， ℓ_j 为指数值（归一化因子）的滚动求和值。中间输出 O_j 的更新方式为: $O_j = e^{m_{j-1}-m_j}O_{j-1} + e^{S_j-m_j}V_j$. 而重缩放因子 $e^{m_{j-1}-m_j}$ 则能在遇到较大数值时对之前的结果进行重新归一化，从而保障数值稳定性。

条件重缩放。第 $e^{m_{j-1}-m_j}O_{j-1}$ 步需要执行向量乘法。我们得到了两个简单的观察结果:

1. 重新缩放 仅当出现 $m_j > m_{j-1}$时才需要调整对齐，也就是说，在找到新的更大数值时。 .

2. 在重缩放过程中我们可以允许存在一定的“松弛度”：仅当 $m_j - m_{j-1} > \tau$时才进行重缩放，其中 τ 为某个阈值（通常设置为$\log_2(256) = 8.0$，对应的重缩放因子为256.0）。只要我们持续记录相关统计信息（即已执行的总缩放量），最终依然能够得到准确的分母，从而得出正确的结果。

在 FlashAttention-4中，我们对算法进行了如下修改:

$$O_j = \begin{cases} e^{m_{j-1}-m_j}O_{j-1} + e^{S_j-m_j}V_j & \text{if } m_j - m_{j-1} > \tau \\ O_{j-1} + e^{S_j-m_{j-1}}V_j & \text{otherwise} \end{cases} \tag{6}$$

当 $m_j - m_{j-1} \le \tau$时，我们会跳过对 m 的更新，继续使用m_{j-1}。这样一来仍能保证结果的正确性，因为在计算结束时，所有累积的值都会通过真实的最大值m_{final} 以及最终的归一化因子 ℓ_{final}进行重新归一化。

$$\text{Output} = \frac{1}{\ell_{\text{final}}}O_{\text{final}}$$

由于最后的归一化步骤能够修正因跳过中间归一化而产生的微小偏差，这种修改在保持数值精确度的同时，大幅减少了数值重归一化操作的次数。

在实际应用中，为避免出现串行单元发散的情况，只要串行单元中的任意一个线程需要归一化处理，我们就会立即执行归一化操作。

3.2 注意力反向传播

3.2.1 Feeds andSpeeds

与前向传播类似，我们首先通过分析矩阵乘法单元（张量核心）、共享内存（SMEM）以及指数运算单元的吞吐量，来研究其轮廓线，从而为我们的内核设计及优化措施提供直观依据。

设**Q** 与**K** 在序列长度维度上的块形状为 $M \times N$，头维度则为 d。我们会分析计算需求与内存流量情况，以此找出性能瓶颈。与前向传播不同，为了简化SMEM循环次数的计算公式，我们假设 $M = N = d = 128$，不过仍保留变量名称以便于理解。

MMA运算。在每次迭代中，后向传播会执行五次矩阵乘累加操作。每次矩阵乘累加操作都涉及一个 $M \times N$ 矩阵、一个 $M \times d$ 矩阵以及一个 $d \times N$ 矩阵（其中哪一个作为输出矩阵会有所不同），因而需要 $2MNd$ 次浮点运算。若采用具有每周期8192次浮点运算次数的张量核心，

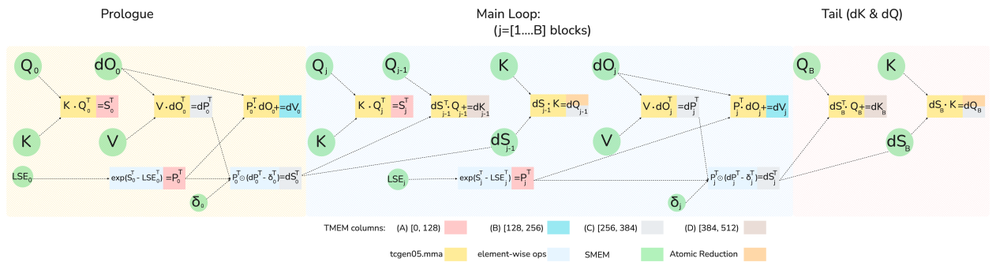


Figure 2: FlashAttention-4 backward computation graph (5 MMA operations + 2 elementwise operations), showing the 1-CTA MMA mode software pipeline order across the prologue, main loop, and tail.

core throughput of 8192 FLOPs per cycle, the total compute time is

$$T_{\text{MMA}} = \frac{10MNd}{8192} \text{ cycles.} \tag{7}$$

Shared memory traffic. Of the five MMAs, three of them – $\mathbf{S}^\top = \mathbf{KQ}^\top$, $\mathbf{dP}^\top = \mathbf{VdO}^\top$, and $\mathbf{dQ} = \mathbf{dSK}$ – are shared-shared (SS) operations where both operands are read from shared memory, while two – $\mathbf{dV} = \mathbf{P}^\top\mathbf{dO}$ and $\mathbf{dK} = \mathbf{dS}^\top\mathbf{Q}$ – are tensor-shared (TS) operations where operand A is read from tensor memory and operand B from shared memory. The SS MMAs read in total $2Md + 3Nd + MN$ elements from shared memory, while the TS MMAs read in total $2Md$ elements from shared memory. At a shared memory bandwidth of 128 bytes per cycle and with each element being 2 bytes (bf16), this contributes

$$T_{\text{smem,MMA}} = \frac{4Md + 3Nd + MN}{64} \text{ cycles.} \tag{8}$$

Additionally, the algorithm writes the intermediate gradient $\mathbf{dS}$ of size $M \times N$ to shared memory in bf16, requiring $2MN$ bytes or $MN/64$ cycles. The gradient $\mathbf{dQ}$ of size $M \times d$ is written in fp32 (4 bytes per element) to shared memory, then read back via TMA for the reduction, totaling $8Md$ bytes of shared memory traffic or $Md/16$ cycles.

The total shared memory access time (T_{smem}) is therefore

$$\frac{4Md + 3Nd + MN}{64} + \frac{MN}{64} + \frac{Md}{16} \text{ cycles.} \tag{9}$$

Exponential unit. The exponential unit computes elementwise operations (exponentials, logarithms, and related nonlinear functions) required for the softmax and its gradient. The backward pass requires exponential operations on the $M \times N$ values (corresponding to the attention matrix $\mathbf{S}$ and related terms). With a throughput of 16 operations per cycle, the exponential unit requires

$$T_{\text{exp}} = \frac{MN}{16} \text{ cycles.} \tag{10}$$

Table 3 summarizes the analysis for the typical tile configuration $M = N = d = 128$. The shared memory traffic time of 3328 cycles exceeds both the MMA compute time (2560 cycles) and the exponential unit time (1024 cycles), indicating that shared memory bandwidth is the primary bottleneck, though less severely than global memory traffic would suggest. This motivates our kernel design to maximize overlap between MMA operations and other computations to hide shared memory latency.

3.2.2 New pipeline to overlap matmul and softmax

The backward pass in flash attention performs five MMA operations, corresponding to recomputing $\mathbf{S}$, and the two gradient computations induced by $\mathbf{QK}$ (which yields $\mathbf{dQ}$ and $\mathbf{dK}$) and $\mathbf{PV}$ (which

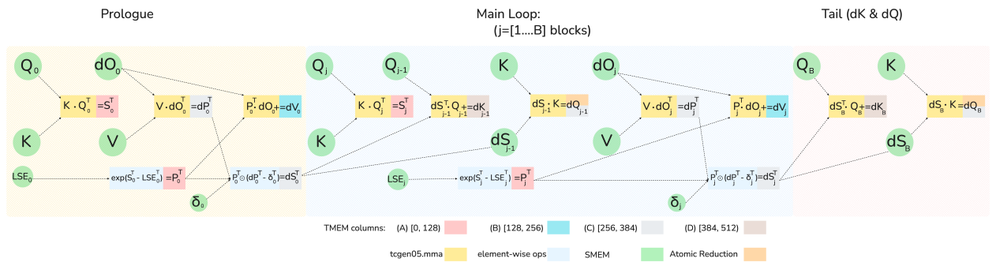


图2：FlashAttention-4的逆向计算图（包含5次MMA运算及 + 2 次元素级运算），展示了在序言部分、主循环以及尾部阶段中1-CTA MMA模式软件流水线的执行顺序。

那么总的运算时间即为

$$T_{\text{MMA}} = \frac{10MNd}{8192} \text{ cycles.} \tag{7}$$

Shared memory traffic. Of the five MMAs, three of them – $\mathbf{S}^\top = \mathbf{KQ}^\top$, $\mathbf{dP}^\top = \mathbf{VdO}^\top$, and $\mathbf{dQ} = \mathbf{dSK}$ – are shared-shared (SS) operations where both operands are read from shared memory, while two – $\mathbf{dV} = \mathbf{P}^\top\mathbf{dO}$ and $\mathbf{dK} = \mathbf{dS}^\top\mathbf{Q}$ – are tensor-shared (TS) operations where operand A is read from tensor memory and operand B from shared memory. The SS MMAs read in total $2Md + 3Nd + MN$ elements from shared memory, while the TS MMAs read in total $2Md$ elements from shared memory. At a shared memory bandwidth of 128 bytes per cycle and with each element being 2 bytes (bf16), this contributes

$$T_{\text{smem,MMA}} = \frac{4Md + 3Nd + MN}{64} \text{ cycles.} \tag{8}$$

此外，该算法会将大小为 $M \times N$ 的中间梯度$\mathbf{dS}$ 以BF16格式写入共享内存，这需要 $2MN$字节的内存空间或 $MN/64$次循环。大小为 $M \times d$的梯度$\mathbf{dQ}$则以FP32格式（每个元素4字节）写入共享内存，随后通过TMA操作将其读回以进行求和运算，总共会产生 $8Md$字节的内部内存流量或需要 $Md/16$ 次循环。

因此，总共享内存访问时间（T_{smem}）为

$$\frac{4Md + 3Nd + MN}{64} + \frac{MN}{64} + \frac{Md}{16} \text{ cycles.} \tag{9}$$

指数运算单元。该指数运算单元负责执行softmax函数及其梯度计算所需的逐元素运算（指数值、对数以及相关的非线性函数）。后向传播过程中需要对 $M \times N$ 值（对应于注意力矩阵$\mathbf{S}$及相关参数）进行指数运算。由于每个周期可完成16次运算，因此该指数运算单元需要

$$T_{\text{exp}} = \frac{MN}{16} \text{ cycles.} \tag{10}$$

表3汇总了 $M = N = d = 128$配置下的分析结果。3328次循环次数的共享内存流量时间超过了MMA运算所需的时间（2560次循环）以及指数运算单元的处理时间（1024次循环），这表明共享内存的带宽是主要的瓶颈，不过其严重程度尚不及全局内存流量所显示的那样高。这一情况促使我们在内核设计中尽可能增加MMA运算与其他计算任务之间的重叠度，以此来抵消共享内存带来的延迟影响。

3.2.2 用于实现matmul与softmax函数重叠计算的新流程

在FlashAttention机制中的后向传播过程会执行五次MMA运算，这些运算实际上相当于重新计算$\mathbf{S}$的值；此外还会产生两次梯度计算，分别由$\mathbf{QK}$ （进而得出$\mathbf{dQ}$ 和$\mathbf{dK}$）以及$\mathbf{PV}$ 引发。

Resource	Cycles ($N = d = 128$) **1-CTA** ($M = 128$)	Cycles ($N = d = 128$) **2-CTA** ($M = 256$)
MMA compute	2560	2560
Shared memory (MMA operands)	2048	1536
Shared memory (**dS** write)	256	256
Shared memory (**dS** *DSMEM*)	0	384
Shared memory (**dQ** write + read)	1024	512
Total shared memory	**3328**	**2688**
Exponential unit	1024	1024

Table 3: Roofline analysis for the attention backward pass with $M = N = d = 128$. Shared memory traffic is the bottleneck, exceeding MMA compute time by approximately 30%. In the 2-CTA setting with $M = 256$ and $N = d = 128$ (with an exception to the **dQ** mma, with $M = N = 128$ and $d = 256$), shared memory traffic exceeds MMA compute time by approximately 5%.

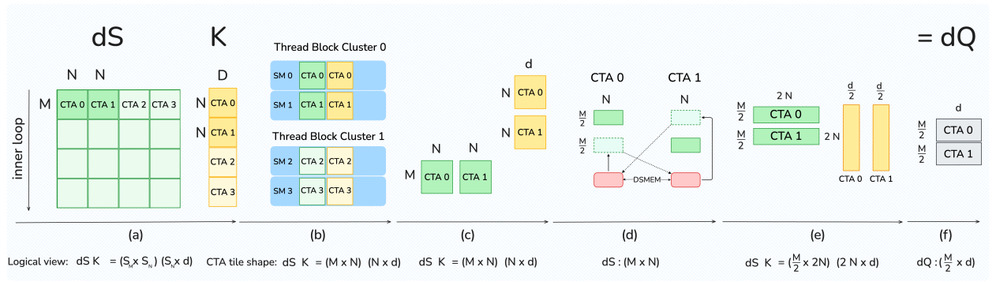


Figure 3: In the 2-CTA backward dQ step, the CTA pair uses DSMEM to exchange half of the dS tile so each CTA forms an $(\frac{M}{2} \times 2N)$ operand and can run a CTA-pair UMMA with a doubled reduction.

yields **dP** and **dV**) respectively. In FA-3, accumulators are stored in registers and registers are a limited resource. This imposes significant ordering constraints that—in effect—serialize the compute graph, i.e., compute $\mathbf{S}, \mathbf{dP}, \mathbf{dV}, \mathbf{dQ}, \mathbf{dK}$, with only the TMA load running significantly out of turn. In addition to that, the algorithm is similar: It iterates along the KV sequence length dimension and computes values transposed w.r.t. the forward pass, since that is the layout that the **dV** and **dK** gradient calculations require to read one of their operands from tensor memory. **dQ** accumulates through the atomics.

In FA-4, TMEM enables additional schedules compared to FA-3 that provide significant overlap between MMA and non-MMA operations. Specifically, just as with the forward pass, we are trying to hide the latency of the softmax calculation. In FA-3, the softmax computation overlaps with the MMA for **dP**. From the previous section, we know that on Blackwell we need at least two MMA operations to run concurrently.

We achieve this by using the **dQ** and **dK** MMA's of the previous iteration. This requires careful management of shared memory and tensor memory resources between loading, MMA, compute, and reduction operations. In particular, note that we do not have sufficient tensor memory to fit five accumulator tiles. At most, four tiles of 128 by 128 elements can fit and **dV** and **dK** accumulate, and thus cannot share their space. In our implementation, we have **S** and **P** share one of the tmem blocks (at offset 0) and we have **dP**, **dS** and **dQ** share the other one. We demonstrate the computational graph of the FA-4 bwd in Figure 2.

资源	循环次数：$N = d = 128$ **1-CTA** ($M = 128$)	循环次数：$N = d = 128$ **2-CTA** ($M = 256$)
MMA运算	2560	2560
共享内存（用于MMA操作数）	2048	1536
共享内存（**dS**写入操作）	256	256
共享内存（**dS** *DSMEM*）	0	384
共享内存（**dQ**写入及 + 读取操作）	1024	512
总共享内存量	**3328**	**2688**
指数运算单元	1024	1024

表3：包含 $M = N = d = 128$的注意力机制后向传播过程的性能边界分析结果。其中，共享内存流量是主要的瓶颈，其消耗时间大约比MMA运算时间多出30%。而在采用 $M = 256$ 和 $N = d = 128$ 的2-CTA模式下（**dQ** 相关的MMA运算以及包含 $M = N = 128$ 和$d = 256$的情况除外），共享内存流量的占比则使总耗时比MMA运算时间多出约5%。

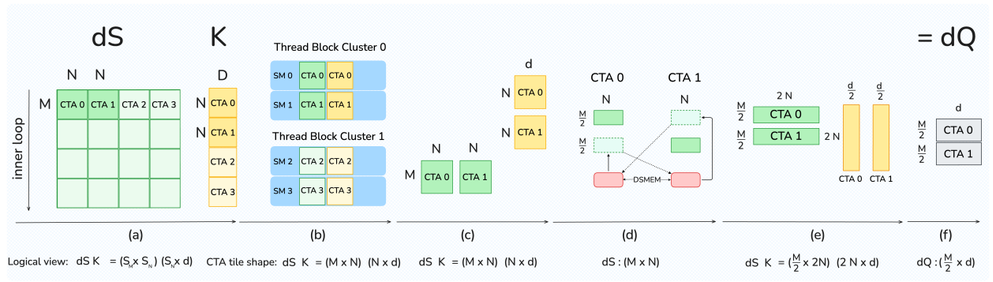


图3：在2-CTA模式反向 dQ 步骤中，CTA对会利用SMEM来交换 dS 瓦片的一半数据，从而使每个CTA都能构成一个$(\frac{M}{2} \times 2N)$操作数，并能够执行具有双倍归约操作的CTA对UMMA运算。

yields **dP** and **dV**) respectively. In FA-3, accumulators are stored in registers and registers are a limited resource. This imposes significant ordering constraints that—in effect—serialize the compute graph, i.e., compute $\mathbf{S}, \mathbf{dP}, \mathbf{dV}, \mathbf{dQ}, \mathbf{dK}$, with only the TMA load running significantly out of turn. In addition to that, the algorithm is similar: It iterates along the KV sequence length dimension and computes values transposed w.r.t. the forward pass, since that is the layout that the **dV** and **dK** gradient calculations require to read one of their operands from tensor memory. **dQ** accumulates through the atomics.

在FA-4中，相较于FA-3，TMEM支持更丰富的调度策略，从而实现MMA运算与非MMA运算之间的显著重叠。具体而言，与前向传播类似，我们同样试图隐藏softmax计算带来的延迟。在FA-3中，softmax运算与**dP**相关的MMA运算存在重叠。从上一节可知，在Blackwell架构上至少需要两个MMA运算才能同时执行。

We achieve this by using the **dQ** and **dK** MMA's of the previous iteration. This requires careful management of shared memory and tensor memory resources between loading, MMA, compute, and reduction operations. In particular, note that we do not have sufficient tensor memory to fit five accumulator tiles. At most, four tiles of 128 by 128 elements can fit and **dV** and **dK** accumulate, and thus cannot share their space. In our implementation, we have **S** and **P** share one of the tmem blocks (at offset 0) and we have **dP**, **dS** and **dQ** share the other one. We demonstrate the computational graph of the FA-4 bwd in Figure 2.

3.2.3 2-CTA backward pass: Reducing shared memory traffic and global atomic adds

Even with improved pipelining and with two of the ten GEMM operands resident in tensor memory, shared memory bandwidth still dominates the backward pass. Across the five GEMMs, the remaining eight BF16 operands are loaded from shared memory to feed the tensor cores, and this shared memory traffic incurs about 30% more cycles than the tensor core compute. To further mitigate this bottleneck, we use the 2-CTA MMA mode introduced by Blackwell, in which the output accumulator is partitioned in the M dimension. With an MMA tile shape of $M = 256$ and $N = K = 128$, the two CTAs act as a single larger tile: each CTA loads and stages half of operand B and keeps only its own accumulator slice.

Shared memory traffic. With five GEMMs in the backward pass, we use MMA tile shape of $M = 256$ and $N = K = 128$, which roughly halves the shared memory traffic for operand B. In the FlashAttention backward pass, each CTA holds a fixed KV tile (parallelize the outer loop across N CTAs) and streams over M tiles in the inner loop. The accumulation of **dQ** is a reduction in the KV sequence in the outer loop, but the 2-CTA MMA only splits the output tile, not the reduction axis, and the reduction dimension of the **dQ** MMA is N, which is naturally split across the CTA pair. As a result, each CTA still needs the full reduction for the rows it owns. To address this conflict on the reduction axis, we use distributed shared memory (DSMEM) to exchange half of the dS between the two CTAs since they are in the same cluster. This approach repacks **dS** so that it is partitioned along the non-reduction axis, with each CTA owning its $\frac{M}{2}$ rows and holding the full $2N$ reduction. As a result, the per CTA **dQ** MMA tile shape is $(\frac{M}{2}, 2N)(2N, d)$ and accumulates a tile $(\frac{M}{2}, d)$ in tensor memory. In 2-CTA MMA mode, the MMAs for **S**, **dP**, **dV** and **dK** run with the tile $M = 256$, while **dQ** uses $M = 128$ but with double reduction $2N = 256$. We then reordered the software pipeline with respect to the 1-CTA variant to hide the DSMEM latency. We compute **dP** for the current tile before computing **dQ** for the tile of the previous iteration. The **dQ** tile is small enough to fit in TMEM alongside **P**, reusing the same TMEM region as **S**, so we no longer reuse the same TMEM region for **dP** and **dQ** as we do in the 1-CTA mode. With this new pipelining ordering, we can compute the element-wise **dS** of the current tile in parallel with the **dQ** MMA of the tile from the previous iteration. See Figure 3 for an illustration of how the **dQ** step is decomposed.

dQ atomic adds. A complementary benefit of this **dQ** decomposition is that it halves the number of global atomic reductions. Atomic updates introduce nondeterminism and are expensive as they occur in every iteration of the inner loop. Consequently, each CTA writes only half of the **dQ** tile and performs half as many global atomic reductions as the 1-CTA counterpart.

3.2.4 Deterministic backward pass

Our backward kernel introduces nondeterminism for the gradient computation due to inter-CTA reductions in global memory (affecting **dQ** in general, and **dK**/**dV** in the case of GQA). To ensure reproducibility and facilitate reliable debugging during training, we also provide a deterministic execution mode. The standard solution, which we also adopt, is to serialize the global reductions using a semaphore lock. Specifically, each CTA writing to a common **dQ** tile must acquire the lock according to a predefined order, perform its reduction, and then release the lock by incrementing the semaphore counter.

This lock-based approach impacts performance for two main reasons: (1) issuing the memory fence to ensure device-wide visibility of the semaphore write (required for correct acquire–release semantics), and (2) introducing stalls as each CTA waits for previous CTAs reducing on a common **dQ** tile to complete. In load-imbalanced situations, a naive choice of CTA order can severely degrade performance. In general, we do CTA swizzling in the head and batch dimensions to reduce stalls

3.2.3 2-CTA模式下的后向传播：降低共享内存流量与全局原子加法操作

即便通过优化流水线处理，并且将十个GEMM操作数中的两个存储在张量内存中，共享内存的带宽依然在后向传播过程中占据主导地位。在五个GEMM运算中，剩余的八个BF16格式操作数需要从共享内存加载以供给张量核心使用，而这种内存流量所导致的循环次数大约比张量核心本身的计算所需循环次数多出30%。为进一步缓解这一瓶颈，我们采用了Blackwell架构所引入的2-CTA MMA模式，在该模式下输出累加器会在M维度上进行划分。当MMA的块形状为 $M = 256$ 和 $N = K = 128$时，两个CTA会共同构成一个更大的块：每个CTA负责加载并暂存操作数B的一半，并仅保留属于自己的累加器部分。

Shared memory traffic. With five GEMMs in the backward pass, we use MMA tile shape of $M = 256$ and $N = K = 128$, which roughly halves the shared memory traffic for operand B. In the FlashAttention backward pass, each CTA holds a fixed KV tile (parallelize the outer loop across N CTAs) and streams over M tiles in the inner loop. The accumulation of **dQ** is a reduction in the KV sequence in the outer loop, but the 2-CTA MMA only splits the output tile, not the reduction axis, and the reduction dimension of the **dQ** MMA is N, which is naturally split across the CTA pair. As a result, each CTA still needs the full reduction for the rows it owns. To address this conflict on the reduction axis, we use distributed shared memory (DSMEM) to exchange half of the dS between the two CTAs since they are in the same cluster. This approach repacks **dS** so that it is partitioned along the non-reduction axis, with each CTA owning its $\frac{M}{2}$ rows and holding the full $2N$ reduction. As a result, the per CTA **dQ** MMA tile shape is $(\frac{M}{2}, 2N)(2N, d)$ and accumulates a tile $(\frac{M}{2}, d)$ in tensor memory. In 2-CTA MMA mode, the MMAs for **S**, **dP**, **dV** and **dK** run with the tile $M = 256$, while **dQ** uses $M = 128$ but with double reduction $2N = 256$. We then reordered the software pipeline with respect to the 1-CTA variant to hide the DSMEM latency. We compute **dP** for the current tile before computing **dQ** for the tile of the previous iteration. The **dQ** tile is small enough to fit in TMEM alongside **P**, reusing the same TMEM region as **S**, so we no longer reuse the same TMEM region for **dP** and **dQ** as we do in the 1-CTA mode. With this new pipelining ordering, we can compute the element-wise **dS** of the current tile in parallel with the **dQ** MMA of the tile from the previous iteration. See Figure 3 for an illustration of how the **dQ** step is decomposed.

dQ原子加法操作。这种**dQ** 分解方法的另一个优势在于它能将全局原子化归约操作的次数减少一半。由于原子更新会在内层循环的每一次迭代中执行，因此不仅会引入非确定性，还会带来较高的计算开销。正因如此，每个CTA结构仅需要写入**dQ** 分块中的一半内容，其执行的全局原子化归约操作次数也仅为1-CTA结构的一半。

3.2.4 确定性反向传播

我们所设计的反向内核由于在全局内存中进行跨CTA的求和操作，会在梯度计算过程中引入非确定性（通常会影响<span style id='1'>dQ，而在GQA场景下则会影响到<span style id='3'>dK/<span style id='5'>dV）。为确保训练过程的可重复性并便于进行可靠调试，我们还提供了确定性的执行模式。我们同样采用的标准解决方案是通过信号量锁来对全局求和操作进行串行化处理。具体而言，每个向公共<span style id='7'>dQ区域写入数据的CTA都必须按照预定义的顺序获取锁，完成求和运算后，再通过增加信号量计数器来释放锁。

这种基于锁的实现方式会从两个主要方面影响性能：其一，需要发出内存屏障以确保信号量写入操作在整个设备范围内可见（这是实现正确的获取–释放语义所必需的）；其二，由于每个协作线程数组都需要等待之前那些正在处理同一**dQ** 数据块的协作线程数组完成操作，因此会引发停顿。在负载不均衡的情况下，若随意选择协作线程数组的处理顺序，性能将会严重下降。为了解决这一问题，我们通常会对头部维度与批次维度中的协作线程数组进行数据重排操作，以此减少停顿现象。

(up to L2 cache capacity, cf. Section 3.3). For causal masking, we additionally launch KV blocks in descending order, traverse query blocks in ascending order starting from the diagonal, and order the **dQ** reductions by descending query block index. This "shortest-processing-time-first" (SPT) schedule ensures that no CTA is stalled on its first **dQ** write.

3.3 Scheduling

In many situations, such as with causal masking or variable sequence length (varlen), the attention kernel is naturally load-imbalanced – SMs are assigned worktiles whose mainloops differ in length, since some worktiles require more loads and MMAs than others. Furthermore, we can choose the order in which SMs process tiles, for instance by defining a preferred linearization of the grid coordinates. Abstracting away any specific features of attention, we can then apply general results on makespan minimization for identical parallel processors to our context. In particular, in FlashAttention-4, we use the classical idea of longest-processing-time-first (LPT) scheduling [9]. We emphasize that the way in which we apply this idea works across all GPU architectures and was also validated as an improvement to FlashAttention-3 on Hopper GPUs.

LPT for causal masking. The standard attention grid is given by (mblocks, heads, batches) and is computed in increasing order left-to-right. But scores are masked out above the diagonal, so for fixed head and batch, SMs end up inefficiently processing worktiles from shortest to longest. On the other hand, a naive LPT order is also suboptimal, since for different batches, mainloop KV loads won't hit in L2 cache, and loading all KV heads first can thrash the L2 cache if they exceed its capacity. Instead, we always process batches as the outermost dimension, and swizzle over heads. This means that we divide heads into sections that don't overflow L2 cache; the tile scheduler then traverses the grid by heads per section, mblocks in reverse order, sections, and finally batches. In particular, for MQA or GQA, we always traverse all query heads per KV head before varying over mblocks. Empirically, we validate that this LPT order is highly effective; for example, for BF16 and head dimension 128 we obtain 4-8% FLOPS gain for MHA and 7-14% for MQA 8 as measured on an H200 GPU.

LPT for variable sequence length. For varlen, we also have to contend with load imbalance due to variation among batches. For instance, in decode workloads, different batches might attend to different amounts of context, and in mixed or continuous batching, some batches might be prefill while others are decode. The list of query and KV sequence lengths per batch is typically stored as attention metadata on device, and the standard varlen attention kernel reads these integers at runtime while processing batches in increasing order. However, the given batch order may be arbitrarily suboptimal with respect to load balancing – for example, we could have shorter square prefills followed by long-context decodes. To ameliorate this, we can enforce an LPT order by launching a preprocessing kernel to sort batches according to their maximum per-worktile execution time, writing out the additional metadata of a virtual to actual batch index mapping that will be subsequently read back into the attention kernel in order to traverse batches in sorted order. This metadata can be cached and thus results in no performance loss from sorting.

4 Language and Framework

We write FlashAttention-4 entirely in CuTe-DSL [21], embedded in Python, without any component in CUDA C++. The CuTe-DSL compiler takes the source code in Python, lowers to PTX, then uses the PTX compiler (ptxas) to finally produce the assembly code (SASS).

Full expressivity with clean abstractions. The CuTe-DSL programming model is isomorphic to CUTLASS C++, ensuring that FlashAttention-4 retains the full expressivity of low-level GPU programming while benefiting from the productivity gains of meta-programming in Python instead of C++ and fast JIT compilation times. CuTe-DSL provides direct access to PTX as an

（处理数量受L2缓存容量限制，详见第3.3节。）针对因果掩码机制，我们还会以降序方式启动KV块，从对角线位置开始按升序遍历查询块，并根据查询块的索引大小以降序排列**dQ** 值的缩减操作。这种“处理时间最短者优先”的SPT调度策略能够确保没有任何协作线程数组会在其首次执行**dQ** 写入操作时出现停顿。

3.3 调度机制

在许多情况下，比如存在因果掩码机制或序列长度不固定的情形时，注意力内核往往会出现负载不平衡的问题——由于部分工作块需要比其他块更多的数据加载与多精度算术单元运算，因此分配给流式多处理器的各个工作块的主循环长度也会存在差异。此外，我们还可以通过定义网格坐标的优选线性化顺序等方式，来决定流式多处理器处理这些工作块的先后顺序。在忽略注意力机制的特定特征后，我们便可以将针对相同并行处理器的任务完成时间最小化问题的通用结论应用到当前场景中。特别是在FlashAttention-4中，我们采用了“处理时间最长的任务优先”这一经典的LPT调度策略 [9]。需要强调的是，我们运用这一策略的方法适用于所有GPU架构，并且在Hopper架构的GPU上也被验证能够提升FlashAttention-3的性能。

LPT for causal masking. The standard attention grid is given by (mblocks, heads, batches) and is computed in increasing order left-to-right. But scores are masked out above the diagonal, so for fixed head and batch, SMs end up inefficiently processing worktiles from shortest to longest. On the other hand, a naive LPT order is also suboptimal, since for different batches, mainloop KV loads won't hit in L2 cache, and loading all KV heads first can thrash the L2 cache if they exceed its capacity. Instead, we always process batches as the outermost dimension, and swizzle over heads. This means that we divide heads into sections that don't overflow L2 cache; the tile scheduler then traverses the grid by heads per section, mblocks in reverse order, sections, and finally batches. In particular, for MQA or GQA, we always traverse all query heads per KV head before varying over mblocks. Empirically, we validate that this LPT order is highly effective; for example, for BF16 and head dimension 128 we obtain 4-8% FLOPS gain for MHA and 7-14% for MQA 8 as measured on an H200 GPU.

适用于序列长度可变场景的LPT算法。在处理序列长度可变的情况时，我们还必须应对因批次数据差异而导致的负载不平衡问题。例如，在解码类任务中，不同的批次可能需要处理不同长度的上下文；而在混合或连续批处理模式下，有些批次可能处于预填充阶段，而另一些则处于解码阶段。每个批次中的查询序列长度和KV序列长度信息通常会作为注意力机制的元数据存储在设备上，标准的序列长度可变注意力内核会在运行时按顺序处理这些批次，并读取相应的整数值。然而，现有的批次处理顺序可能在负载均衡方面并非最优——比如，可能会出现先处理较短长度的预填充数据，随后再处理需要处理长上下文的解码数据的情形。为了解决这一问题，我们可以通过启动一个预处理内核来按照每个工作单元的最大执行时间对批次进行排序，同时生成虚拟批次索引与实际批次索引之间的映射关系这一额外元数据，之后再将该元数据读回注意力内核，从而按排序后的顺序处理批次。这类元数据可以被缓存起来，因此不会因排序操作而带来性能损失。

4 语言与框架

我们完全使用嵌入在Python语言中的CuTe-DSL语言来编写 FlashAttention-4 代码，全程未使用任何CUDA C++相关组件。CuTe-DSL编译器会先将Python源代码转换为PTX格式，再通过PTX编译器（ptxas）最终生成汇编代码（汇编代码）。

通过简洁的抽象实现完整的表达能力。 CuTe-DSL编程模型与CUTLASS C++是同构的，这使得 FlashAttention-4 既能保留低级GPU编程的全部表达能力，又能借助Python语言的元编程优势以及快速的即时编译技术，提升开发效率。此外，CuTe-DSL还提供了对PTX的直接访问功能，可作为

escape hatch, allowing developers to implement any functionality they need without framework limitations. For example, we leverage custom PTX sequences for operations not yet fully exposed in CuTe-DSL APIs (though these will be integrated in future releases), demonstrating that our framework does not constrain developers to a limited subset of GPU capabilities.

Fast compilation through JIT. Compile time has been a bottleneck in past FlashAttention implementations, due to complex C++ template metaprograms. By embedding CuTe-DSL in Python with just-in-time (JIT) compilation, FlashAttention-4 achieves faster build times compared to traditional C++ template-based approaches. As shown in Table 4, FlashAttention-4 reduces compile time by 20-30× compared to FlashAttention-3. This rapid iteration cycle significantly improves developer productivity, enabling faster experimentation and debugging during kernel development.

Method	Forward pass	Backward pass
FlashAttention-3	55s	45s
FlashAttention-4	2.5s	1.4s
Speedup	22×	32×

Table 4: Compile time for a single kernel: FA3 (C++ templates) and FA4 (CuTe-DSL). Typically FA2 and FA3 require precompiling hundreds of kernels for different attention variants.

Flexibility and accessibility. The Python based framework has already demonstrated its flexibility in practice: developers have successfully built FlexAttention and block-sparse attention variants on top of FlashAttention-4 without modifying the core framework. By lowering the barrier to entry, our approach enables researchers and engineers with just a few months of GPU programming experience to contribute meaningful extensions without requiring deep expertise in C++ template metaprogramming. This accessibility accelerates innovation and allows the attention mechanism research community to more rapidly explore new algorithmic variants.

Our vision is to provide a comprehensive framework for building all kinds of attention variants with best-in-class performance. Rather than implementing each attention variant from scratch, FlashAttention-4 factors common functionality into independent, composable primitives. Operations such as block-sparse patterns, masking strategies, variable sequence length handling, and work scheduling are all exposed as orthogonal primitives that can be freely combined. This modular design ensures that optimizations and new features benefit all attention implementations built on the framework while still achieving the highest performance by compiling down to efficient GPU kernels.

5 Empirical Evaluation

We evaluate FlashAttention-4 efficiency compared to various open-source and closed-source baselines.

Benchmarking attention. We measure the runtime of FlashAttention-4 across different sequence lengths and head dimensions, comparing it to standard implementations in PyTorch, FlashAttention-2[1], Triton (which uses B200-specific instructions [26]), Gluon (a lower-level GPU programming language with finer control than Triton [25]), and cuDNN (a vendor's library optimized for B200 GPUs). We confirm that FlashAttention-4 is up to 1.3× faster than cuDNN 9.13 and up to 2.7× faster than Triton. FlashAttention-4 reaches up to 1613 TFLOPs/s, approximately 71% of the theoretical maximum TFLOPs/s on B200 GPUs.

[1] FlashAttention-3 does not run on B200

一种应急出口，让开发者无需受框架限制即可实现任何所需的功能。例如，对于那些目前尚未在CuTe-DSL接口中完全提供的操作，我们可以通过自定义的PTX序列来实现（这些功能将来也会被整合进来），这充分证明了我们的框架并不会将开发者束缚在有限的GPU功能范围内。

通过即时编译实现快速编码。在以往的FlashAttention实现中，由于复杂的C++ 模板元程序，编译时间一直是个瓶颈。通过将CuTe-DSL语言嵌入Python并结合即时编译技术，FlashAttention-4 相比传统的基于C++ 模板的方法能够显著缩短构建时间。如表4所示，与 FlashAttention-3相比，它的编译时间可减少20-30× 。如此快速的迭代周期极大提升了开发人员的效率，使得在内核开发过程中能够更迅速地进行实验和调试。

方法	前向传播	后向传播
FlashAttention-3	55s	45s
FlashAttention-4	2.5s	1.4s
加速比	22×	32×

表4：单个内核的编译时间——FA3（C++ 模板）与FA4（CuTe-DSL）。通常情况下，FA2和FA3需要预先编译数百个针对不同注意力机制变体的内核。

高度的灵活性与易用性。这个基于Python语言的框架已在实际应用中展现出出色的灵活性：开发者们无需修改核心框架，便能在 FlashAttention-4 的基础上成功构建出FlexAttention以及块稀疏注意力机制的各类变体。通过降低入门门槛，我们的方法使得那些仅有几个月GPU编程经验的研究人员和工程师也能贡献有价值的扩展功能，而无需具备深厚的C++ 模板元编程专业知识。这样的易用性极大地推动了创新，让注意力机制研究社区能够更快地探索新的算法变体。

我们的愿景是提供一个功能完备的框架，用于构建各类具备业界最佳性能的注意力机制变体。FlashAttention-4并未从零开始实现每一个注意力机制变体，而是将通用功能拆解为独立的、可组合的原始组件。诸如块稀疏模式、掩码策略、可变序列长度处理以及任务调度等操作，都被作为相互独立的原始组件呈现出来，可供用户自由组合使用。这种模块化设计不仅确保了优化措施和新功能能够惠及基于该框架构建的所有注意力机制实现，还能通过编译为高效的GPU内核来保持最高的性能水平。

5 实证评估

我们对比了多种开源及闭源基准方案，对 FlashAttention-4 的效率进行了评估。

对注意力机制进行基准测试。我们测量了 FlashAttention-4 在不同序列长度及头维度下的运行时间，并将其与PyTorch中的标准实现、FlashAttention-2[1]Triton（采用针对B200芯片设计的专用指令 [26]）、Gluon（一种比Triton具有更精细控制能力的低级GPU编程语言 [25]）以及cuDNN（专为B200 GPU优化的厂商库）的结果进行了对比。测试结果表明， FlashAttention-4 的运行速度最快可达cuDNN 9.13版本的1.3× 倍，比Triton快达2.7× 倍。 FlashAttention-4的浮点运算次数最高可达1613 TFLOPs/秒，这一数值约相当于B200 GPU理论最大浮点运算次数的71%。

[1] FlashAttention-3 无法在B200芯片上运行。

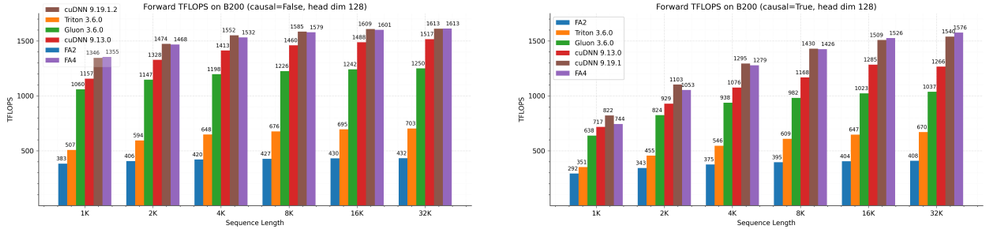


Figure 4: Forward pass TFLOPS on B200 (FP16/BF16) with head dimension 128. Left: non-causal attention. Right: causal attention. FA4 achieves 1.1-1.3× speedup over cuDNN 9.13.0 and 2.1-2.7× over Triton across sequence lengths. Since the initial release of our implementation, newer versions of cuDNN have incorporated many of the techniques described in this paper, yielding similar performance to FA4.

Benchmark settings. We measure runtime on a B200 GPU for different settings (with/without causal mask, head dimensions 64, 128, and (192, 128)) for BF16 inputs. We vary the sequence length as 1k, 2k, ..., 32k, and set batch size so that the total number of tokens is 32k. We set the hidden dimension to 2048, and head dimension to be either 64 or 128 (i.e., 32 heads or 16 heads). For the (192, 128) configuration used in DeepSeek V3 [7], we use 16 heads with 192 query dimensions and 128 key/value dimensions. To calculate the FLOPs of the forward pass, we use $4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}$. With causal masking, we divide this number by 2 to account for the fact that approximately only half of the entries are calculated. To get the FLOPs of the backward pass, we multiply the forward pass FLOPs by 2.5 (since there are 2 matmuls in the forward pass and 5 matmuls in the backward pass, due to recomputation).

5.1 Forward pass

We report forward pass results in Figs. 4 and 5, showing that FlashAttention-4 is 1.1-1.3× faster than cuDNN 9.13 and 2.1-2.7× faster than Triton. For medium and long sequences (4k and above), FlashAttention-4 consistently outperforms all baselines across different head dimensions and causal masking settings. The gains are larger for the causal case, which we attribute to the longest-processing-time first (LPT) scheduler.

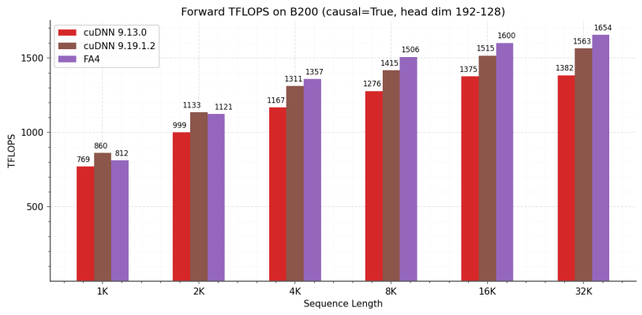


Figure 5: Forward pass TFLOPS comparison between cuDNN and FA4 on B200 (FP16/BF16) with head dimension (192, 128) for causal attention (typically used in DeepSeek V3 architecture)

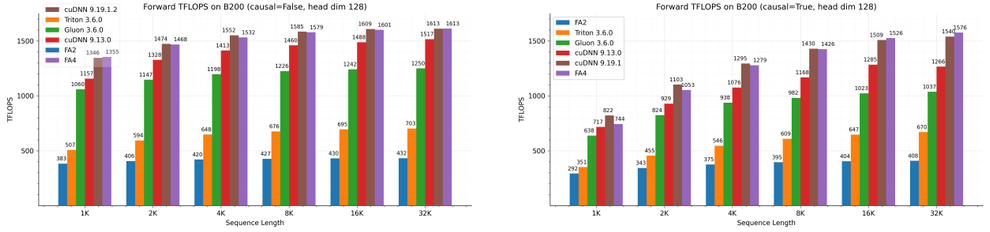


图4：在B200芯片上、采用FP16/BF16格式且头维度为128时的前向传播浮点运算次数。左侧为非因果注意力机制，右侧为因果注意力机制。FA4在各类序列长度下，相较于cuDNN 9.13.0版本的加速比为1.1-1.3× 倍，相较于Triton版本的加速比为2.1-2.7× 倍。自我们的实现首次发布以来，后续版本的cuDNN已引入了本文中描述的许多技术，因而能够达到与FA4相近的性能表现。

Benchmark settings. We measure runtime on a B200 GPU for different settings (with/without causal mask, head dimensions 64, 128, and (192, 128)) for BF16 inputs. We vary the sequence length as 1k, 2k, ..., 32k, and set batch size so that the total number of tokens is 32k. We set the hidden dimension to 2048, and head dimension to be either 64 or 128 (i.e., 32 heads or 16 heads). For the (192, 128) configuration used in DeepSeek V3 [7], we use 16 heads with 192 query dimensions and 128 key/value dimensions. To calculate the FLOPs of the forward pass, we use $4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}$. With causal masking, we divide this number by 2 to account for the fact that approximately only half of the entries are calculated. To get the FLOPs of the backward pass, we multiply the forward pass FLOPs by 2.5 (since there are 2 matmuls in the forward pass and 5 matmuls in the backward pass, due to recomputation).

5.1 前向传播

我们在图4和图5中展示了前向传播的测试结果，数据显示 FlashAttention-4 的运行速度比cuDNN 9.13快1.1-1.3×倍，比Triton快2.1-2.7× 倍。在处理长度为4k及以上的中等及长序列时，FlashAttention-4 在不同的头维度及因果掩码设置下始终优于所有基准算法。其中采用因果掩码机制时的加速效果更为显著，我们认为这是得益于我们采用的“最长处理时间优先”（LPT）调度算法。

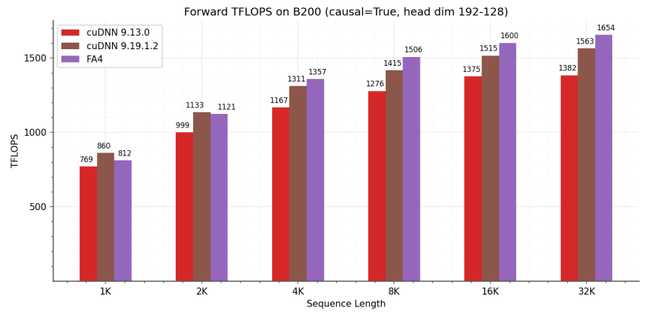


图5：在B200芯片上、采用FP16/BF16格式且头维度分别为192和128的情况下，针对因果注意力机制（通常用于DeepSeek V3架构），cuDNN与FA4在前向传播浮点运算次数上的对比情况。

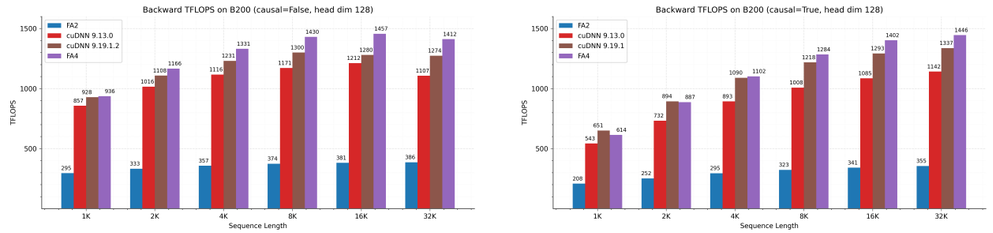


Figure 6: Backward pass TFLOPS on B200 (FP16/BF16) with head dimension 128. Left: non-causal attention. Right: causal attention.

5.2 Backward pass

We report backward pass results in Fig. 6. FlashAttention-4 achieves consistent speedups across long sequence lengths and causal masking, demonstrating the effectiveness of our 2-CTA backward pass.

We also show the performance of the deterministic backward pass in Fig. 7. Our careful swizzling and scheduling results in a much faster deterministic backward pass, getting up to 75% the speed of the nondeterministic backward pass of the 1-CTA backward pass.

Figure 7: Ablations for Deterministic Backward pass on B200 (FP16/BF16) with head dimension 128. Causal attention – SPT, LPT with reverse mblock order, LPT, and naive with no batch/head swizzle.

6 Discussion and Conclusion

FlashAttention-4 addresses asymmetric hardware scaling, where tensor cores are so fast that the dominant bottlenecks shift to shared-memory traffic and exponential throughput, motivating algorithmic and kernel co-design to mitigate these limits. We redesign the pipeline around fully asynchronous MMA to overlap softmax with larger-tiled matmuls and introduce software-emulated exponential and conditional softmax rescaling to reduce non-matmul operations. We leverage tensor memory and 2-CTA MMA mode to reduce shared memory traffic. In addition, 2-CTA enables restructuring the global atomic accumulation, halving the number of global atomic adds.

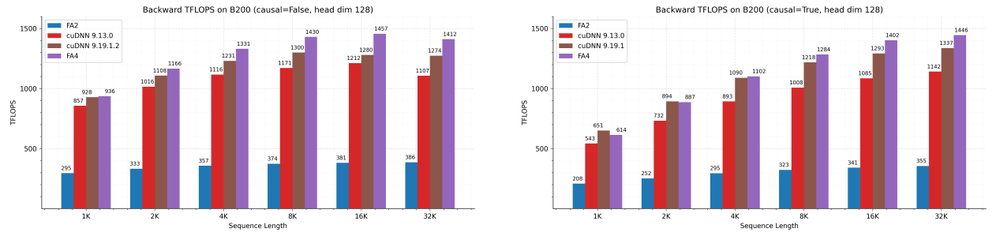


图6：在头维度为128的B200芯片上，采用FP16/BF16格式时的后向传播浮点运算次数。左侧为非因果注意力机制，右侧为因果注意力机制。

5.2 后向传播

图6呈现了后向传播的测试结果。 FlashAttention-4 在各种长序列长度以及不同因果掩码配置下均能保持稳定的加速比，充分证明了我们所设计的2-CTA模式后向传播机制的有效性。

我们在图7中展示了确定性后向传播的性能表现。通过精心设计的数据重排操作与调度机制，该确定性后向传播的速度大幅提升，其速度甚至可达1-CTA结构下非确定性后向传播速度的75%。

图7：在头维度为128的B200芯片上，采用FP16/BF16格式时不同确定性后向传播方法的性能对比。包括因果注意力下的SPT算法、采用逆序m块结构的LPT算法、普通LPT算法，以及未进行批次/头部重排操作的基准方法。

6 讨论与结论

FlashAttention-4针对硬件扩展不对称的问题进行了优化，这类情况下由于张量核心的速度极为快速，主要的瓶颈便转变为共享内存流量以及指数级的处理吞吐量限制，因此有必要通过算法与内核的协同设计来缓解这些约束。我们围绕完全异步的多精度算术单元重新设计了处理流程，使得softmax运算能够与更大块度的矩阵乘法操作并行执行，同时还引入了软件模拟的指数型及条件性softmax重缩放机制，以减少非矩阵乘法操作的数量。此外，我们还利用张量内存以及2-CTA模式下的多精度算术单元，进一步降低了共享内存流量。另外，2-CTA模式还支持对全局原子累加操作进行重构，从而使全局原子加法操作的次数减少一半。

FlashAttention-4 is implemented entirely in CuTe-DSL embedded in Python, preserving low-level control while achieving 20-30× faster compile times than C++ template-based kernels. Although optimized for Blackwell GPUs, some of these algorithms can be extended to other accelerators as compute continues to outpace non-matmul units.

Acknowledgments

We thank Together AI, Meta, xAI, and Princeton Language and Intelligence (PLI) for compute support. We gratefully acknowledge the support of the Schmidt Sciences AI2050 fellowship, the Google ML and Systems Junior Faculty Awards, and the Google Research Scholar program. We want to further thank the following teams at Nvidia: CuDNN, TensorRT-LLM, and Cutlass teams for constant discussions, ideas, and feedback.

References

[1] Michael Bauer, Henry Cook, and Brucek Khailany. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '11, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307710. doi: 10.1145/2063384.2063400. URL `https://doi.org/10.1145/2063384.2063400`.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Richard J Chen, Chengkuan Chen, Yicong Li, Tiffany Y Chen, Andrew D Trister, Rahul G Krishnan, and Faisal Mahmood. Scaling vision transformers to gigapixel images via hierarchical self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16144–16155, 2022.

[4] Sylvain Chevillard, Mioara Joldeş, and Christoph Lauter. Sollya: An environment for the development of numerical codes. In *Mathematical Software – ICMS 2010*, volume 6327 of *Lecture Notes in Computer Science*, pages 28–31. Springer Berlin Heidelberg, 2010.

[5] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023. URL `https://arxiv.org/abs/2307.08691`.

[6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[7] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.

[9] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969. doi: 10.1137/0117039.

[10] Mandy Guo, Joshua Ainslie, David Uthus, Santiago Ontanon, Jianmo Ni, Yun-Hsuan Sung, and Yinfei Yang. Longt5: Efficient text-to-text transformer for long sequences. *arXiv preprint arXiv:2112.07916*, 2021.

FlashAttention-4完全使用嵌入在Python语言中的CuTe-DSL语言实现，在保持低级控制能力的同时，其编译速度比基于C++ 模板的内核快20-30× 倍。尽管这些算法最初是为Blackwell架构的GPU优化的，但随着计算能力持续超越非矩阵乘法单元的性能，其中部分算法也可被扩展到其他类型的加速器上。

致谢

我们感谢Together AI、Meta、xAI以及普林斯顿语言与智能实验室（PLI）在计算资源方面提供的支持。同时，我们也衷心感激Schmidt Sciences AI2050奖学金、Google ML和Systems青年教员奖以及Google研究学者计划的支持。此外，我们还要特别感谢Nvidia的以下团队：CuDNN团队、TensorRT-LLM团队以及Cutlass团队，感谢他们一直以来的交流、宝贵建议与反馈。

参考文献

[1] Michael Bauer、Henry Cook与Brucek Khailany。CudaDMA：通过Warp专用化优化GPU内存带宽。载于*2011*年高性能计算、网络、存储与分析国际会议论文集，SC ' 11，美国纽约州纽约市，2011年。计算机协会出版。ISBN 9781450307710。doi: 10.1145/2063384.2063400。网址：https://doi.org/10.1145/2063384.2063400。

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell 等人。语言模型属于少样本学习器。《神经信息处理系统进展》，33卷：1877–1901页，2020年。

[3] Richard J Chen、Chengkuan Chen、Yicong Li、Tiffany Y Chen、Andrew D Trister、Rahul G Krishnan以及Faisal Mahmood。通过分层自监督学习将视觉变换器模型扩展至千兆像素级图像。《*IEEE/CVF*计算机视觉与模式识别会议论文集》，第16144–16155页，2022年。

[4] Sylvain Chevillard、Mioara Joldeş以及Christoph Lauter所著。《Sollya：一种用于数值代码开发的编程环境》。载于*Mathematical Software – ICMS 2010*，即*Lecture Notes in Computer Science*系列的第6327卷，第28–31页。Springer Berlin Heidelberg出版社，2010年出版。

[5] Tri Dao. FlashAttention-2：兼具更高并行度与更优任务分区的更快注意力机制，2023年。网址：https://arxiv.org/abs/2307.08691。

[6] Tri Dao、Daniel Y. Fu、Stefano Ermon、Atri Rudra以及Christopher R´e。FlashAttention：具有I/O感知能力的快速且内存效率高的精确注意力机制。《神经信息处理系统进展》，2022年。

[7] 深度 Seek-AI。Deepseek-v3技术报告。*arXiv*预印本 *arXiv:2412.19437*，20年 24.

[8] 亚历克谢·多索维茨基、卢卡斯·贝耶、亚历山大·科列斯尼科夫、迪尔克·魏森博恩、翟晓华、托马斯·温特蒂纳、莫斯塔法·德赫加尼、马蒂亚斯·明德勒、格奥尔格·海戈尔德、西尔万·盖利等人。一张图像相当于16×16个词：用于大规模图像识别的变换器模型。载于国际学习表征会议，2020年。

[9] R. L. Graham所著《多进程处理中的计时异常界限研究》。载于*SIAM*应用数学杂志，1969年第17卷第2期，页码416–429，doi编号：10.1137/0117039。

[10] Mandy Guo、Joshua Ainslie、David Uthus、Santiago Ontanon、Jianmo Ni、Yun-Hsuan Sung以及Yinfei Yang。Longt5：一种适用于长序列的高效文本到文本变换器模型。*arXiv*预印本 *arXiv:2112.07916*，2021年。

[11] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.

[12] Jintao Lin, Mingye Luo, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. *arXiv preprint arXiv:2411.10958*, 2024.

[13] Jintao Lin, Jia Zhang, Haofeng Zheng, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. *arXiv preprint arXiv:2410.02367*, 2024.

[14] Jintao Lin, Mingye Luo, Jia Zhang, Zheng Xu, Haofeng Zheng, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention3: Fast attention on blackwell gpus via fp4 quantization. *arXiv preprint arXiv:2505.11594*, 2025.

[15] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Hongyuan Liu, Qiang Wang, and Xiaowen Chu. Dissecting the nvidia hopper architecture through microbenchmarking and multiple level analysis. *arXiv preprint arXiv:2501.12084*, 2025.

[16] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. *Handbook of Floating-Point Arithmetic*. Birkhäuser Boston, 2nd edition, 2018. ISBN 978-3-319-76525-9.

[17] NVIDIA. Nvidia H100 tensor core GPU architecture, 2022.

[18] NVIDIA. CUDA Programming Guide Version 12.4, 2024. URL `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html`.

[19] NVIDIA. Nvidia Blackwell architecture technical brief, 2024. `https://nvdam.widen.net/s/xqt56dflgh/nvidia-blackwell-architecture-technical-brief`.

[20] NVIDIA. cudnn release notes. `https://docs.nvidia.com/deeplearning/cudnn/backend/latest/release-notes.html`, 2025.

[21] Nvidia. Nvidia cutlass documentation, 2025. URL `https://docs.nvidia.com/cutlass/media/docs/pythonDSL/cute_dsl_general/dsl_introduction.html`.

[22] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[23] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL `https://arxiv.org/abs/2407.08608`.

[24] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, et al. Scrolls: Standardized comparison over long language sequences. *arXiv preprint arXiv:2201.03533*, 2022.

[25] Triton Team. Gluon: A lower-level gpu programming language. `https://github.com/triton-lang/triton/blob/main/python/tutorials/gluon/01-intro.py`, 2024.

[11] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.

[12] Jintao Lin, Mingye Luo, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. *arXiv preprint arXiv:2411.10958*, 2024.

[13] Jintao Lin, Jia Zhang, Haofeng Zheng, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. *arXiv preprint arXiv:2410.02367*, 2024.

[14] Jintao Lin, Mingye Luo, Jia Zhang, Zheng Xu, Haofeng Zheng, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention3: Fast attention on blackwell gpus via fp4 quantization. *arXiv preprint arXiv:2505.11594*, 2025.

[15] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Hongyuan Liu, Qiang Wang, and Xiaowen Chu. Dissecting the nvidia hopper architecture through microbenchmarking and multiple level analysis. *arXiv preprint arXiv:2501.12084*, 2025.

[16] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. *Handbook of Floating-Point Arithmetic*. Birkhäuser Boston, 2nd edition, 2018. ISBN 978-3-319-76525-9.

[17] NVIDIA. Nvidia H100 tensor core GPU architecture, 2022.

[18] NVIDIA. CUDA Programming Guide Version 12.4, 2024. URL `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html`.

[19] NVIDIA. Nvidia Blackwell architecture technical brief, 2024. `https://nvdam.widen.net/s/xqt56dflgh/nvidia-blackwell-architecture-technical-brief`.

[20] NVIDIA. cudnn release notes. `https://docs.nvidia.com/deeplearning/cudnn/backend/latest/release-notes.html`, 2025.

[21] Nvidia. Nvidia cutlass documentation, 2025. URL `https://docs.nvidia.com/cutlass/media/docs/pythonDSL/cute_dsl_general/dsl_introduction.html`.

[22] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[23] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL `https://arxiv.org/abs/2407.08608`.

[24] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, et al. Scrolls: Standardized comparison over long language sequences. *arXiv preprint arXiv:2201.03533*, 2022.

[25] Triton Team. Gluon: A lower-level gpu programming language. `https://github.com/triton-lang/triton/blob/main/python/tutorials/gluon/01-intro.py`, 2024.

18

[26] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[26] 菲利普·蒂莱特、龚祥宗与大卫·考克斯共同撰写了《Triton：一种用于分块神经网络计算的中间语言及编译器》一文。该文发表于*ACM SIGPLAN*机器学习与编程语言国际研讨会第三届会议论文集，页码为10–19，出版时间为2019年。

[27] Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N Gomez、Lukasz Kaiser以及Illia Polosukhin共同提出了‘注意力即你所需’这一重要理念。该成果发表在《神经信息处理系统进展》2017年第30卷上。*Advances in neural informationprocessing systems*

19

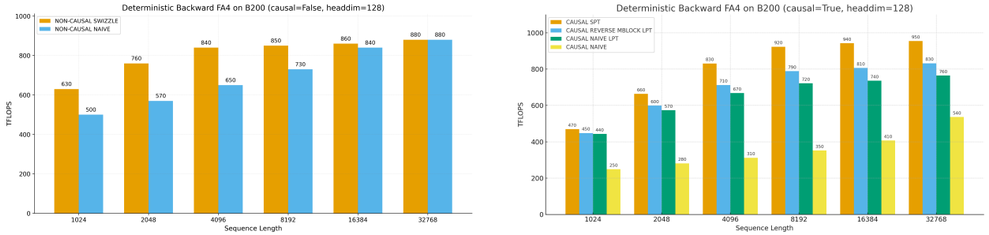


Figure 8: Ablations for Deterministic Backward pass on B200 with head dimension 128. Non-causal attention with batch/head swizzle versus naive. Right: causal attention – SPT, LPT with reverse mblock order, LPT, and naive with no batch/head swizzle.

A Additional Details on Experiments and Benchmarking

A.1 System and libraries

We benchmark the speed on a B100 180GB SXM6 (1000W). We warmup with 5 runs, then repeat the benchmarks 10 times, and take the average timing.

We generally used the latest versions of the libraries at the time of writing (March 2025). Specifically, we use:

- CUDA 13.1
- FlashAttention 2.8.3
- Triton 3.6
- PyTorch 2.10.0
- CuTe-DSL 4.4.1

For cuDNN, in the main paper, we compare to cuDNN 9.13 and the latest version cuDNN 9.19.1.2. Starting from version 9.13 and 9.14 [20], we have worked with the cuDNN team to incorporate some techniques from FlashAttention-4 into cuDNN so that our work can benefit as many practitioners as possible.

A.2 Backward Deterministic non-causal

For completeness, we also include performance numbers for the deterministic backward kernel without causal masking in Fig. 8, side by side with causal masking.

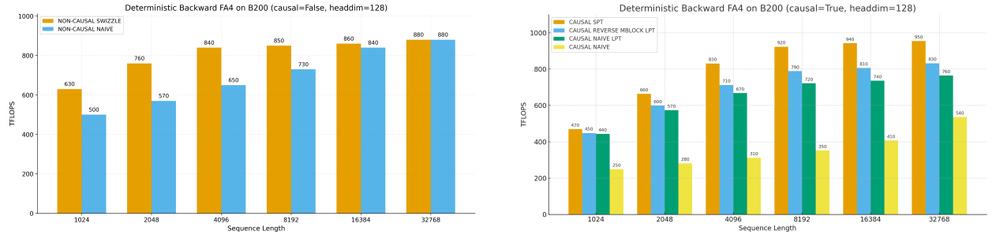


图8：在B200芯片上、头维度为128的情况下对确定性反向传播所进行的消融实验。对比了采用批次/头部重排的非因果注意力与朴素实现；右侧则为因果注意力——包括SPT算法、采用逆m块顺序的LPT算法、普通LPT算法，以及未进行批次/头部重排的朴素实现。

A 实验与基准测试的详细信息

A.1 系统与编程库

我们在配备180GB内存的B100芯片及SXM6插槽（功耗为1000瓦）的硬件环境下对算法速度进行了测试。首先通过5次运行完成预热阶段，随后重复进行10次测试，并最终计算出平均耗时。

在撰写本文时（2025年3月），我们主要使用了当时最新的各类编程库版本。具体而言，我们使用的是：

- CUDA 13.1
- FlashAttention 2.8.3
- Triton 3.6版本
- PyTorch 2.10.0版本
- CuTe-DSL语言 4.4.1版本

在涉及cuDNN的部分中，主论文中我们将该框架与cuDNN 9.13版本以及最新的cuDNN 9.19.1.2版本进行了对比。从9.13版本开始，我们还与cuDNN团队合作，将 FlashAttention-4 中的部分技术整合到cuDNN中，以便让尽可能多的从业者从中受益。

A.2 后向确定性非因果

为保证信息的完整性，我们在图8中同时展示了未应用因果掩码机制的确定性后向内核的性能数据，以及应用了因果掩码机制后的对应性能数据。